

[Организация]
[Адрес, город, почтовый индекс]
Тел. [Телефон]
Факс [Факс]
[Веб-сайт]

РАЗРАБОТКА ТРЕБОВАНИЙ К ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ

23.07.2019

Краткая выжимка из учебника К. Вигерса «Разработка требований к программному обеспечению». Использовать в качестве памятки.

ОГЛАВЛЕНИЕ

Оглавление	2
Уровни и типы требований	7
Бизнес-уровень	7
Бизнес-требования	7
Бизнес правила	7
Пользовательский уровень	8
Пользовательские требования	8
Атрибуты качества (нефункц.)	8
Функциональный уровень	8
Функциональные требования	8
Системные требования	8
Внешние интерфейсы (нефункц.)	8
Ограничения (нефункц.)	8
Общие сведения	9
Требования к проекту	9
Разработка и управление требованиями	9
Разработка требований	9
1. Выявление	9
2. Анализ	10
3. Документирование	10
4. Утверждение	10
Управление требованиями	10
Достижение соглашения	11
Приемы и методы разработки и управления требованиями	11
Связь с методологиями разработки	11
Приемы разработки требований	12
Приемы управления требованиями	13
Разработка требований	14
Общий план разработки требований	14
Определение бизнес-цели	14
Общие сведения	14
Шаблон документа о концепции и границах	14
Способы представления границ проекта	15

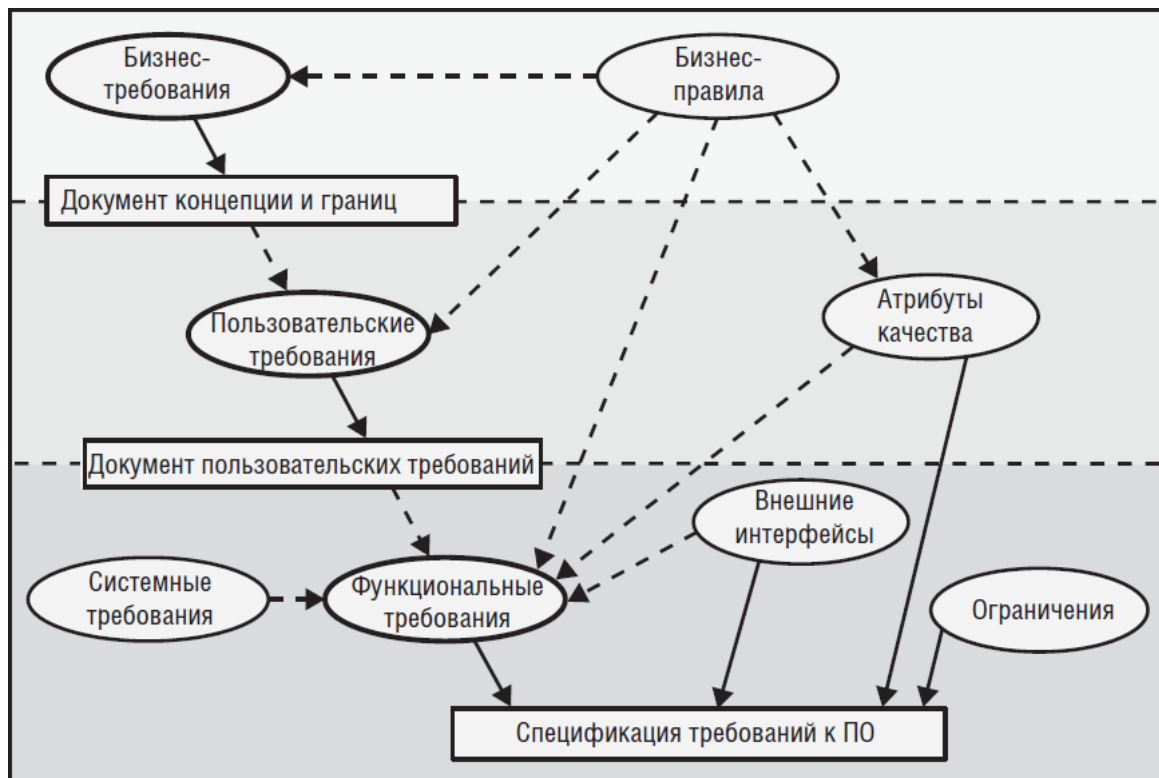
Определение классов пользователей	15
Классификация пользователей	15
Определение классов	16
Представители пользователей	16
Сторонник продукта	16
Разрешение противоречий	18
Выявление требований	18
Методы выявления требований	18
Планирование выявления требований	19
Подготовка к выявлению требований	19
Выявление требований	20
После выявления требований	20
Классификация требований	20
Окончание сбора требований	21
Поиск упущенных требований	21
Дополнительно	21
Варианты использования	21
Общее	21
Пример диаграммы UML	22
Описание варианта использования	22
Макроварианты	23
Методики определения вариантов использования	24
Переход к функциональным требованиям	24
Подводные камни	24
Приоритеты	25
Бизнес-правила	25
Классификация бизнес-правил	25
Документирование бизнес-правил	26
Выявление бизнес-правил	26
Документирование требований	27
Шаблон SRS	27
Общие замечания	28
Атрибуты хорошей документации	28
Номенклатура требований	28
Написание идеальных требований	29
Общая часть	29
Характеристики идеального набора требований	29

Язык и стиль	29
Моделирование требований	29
Требования к данным	29
Моделирование данных: DFD и ERD	29
Словарь данных	30
Анализ данных	31
Спецификация отчетов	31
Панели мониторинга	31
Атрибуты качества ПО	32
Введение	32
Первичное изучение атрибутов	33
Внешние атрибуты качества	34
Внутренние атрибуты качества	39
Язык Planguage	41
Связь атрибутов	41
Переход от атрибутов качества к технической информации	41
Ограничения	42
Использование прототипов	42
Назначение	42
Классификация	42
Создание интерфейса через прототипы	43
Риски при создании прототипа	43
Расстановка приоритетов требований	44
Введение	44
Методы	44
Утверждение требований	46
Введение	46
Экспертиза	46
Этапы экспертизы	46
Контрольный лист дефектов	48
Советы по рецензированию	48
Тестирование требований	48
Критерии приемки	49
Повторное использование требований	49
Три измерения	49
Типы информации для повторного использования	50
Схемы требований	51

От требований к разработке	52
Управление требованиями	54
Основные документы процесса управления	54
Структура процесса управления	54
Управление версиями	54
Управление изменениями	55
Методы управления объемом требований	55
Правила внесения изменений	55
Шаблон описания процесса управления требованиями	55
Совет по внесению изменений	57
Измерение активности внесения изменений	57
Анализ изменений	57
Проблемы с требованиями	59
Управление требованиями в Agile	60
Отслеживание состояний требований	60
Отслеживание связей требований	62
Общая схема	62
Плюсы отслеживания	62
Инструменты для отслеживания связей требований	62
Оценка рисков	64
Риски при разработке требований	64
На стадии выявления требований	64
На стадии анализа	64
На стадии документирования	64
На стадии утверждения	64
Риски при управлении требованиями	64
Приложение А: Моделирование требований	66
Unified Modeling Language, UML	66
Классификация диаграмм	66
State (machine) Diagram	66
Sequence diagram	66
Class diagram	66
ПО для работы с UML	66
Icam DEFinition for Function Modeling, IDEF	67
IDEF0	67

Business Process Modeling Notation, BPMN	67
Другие варианты	67
Data Flow Diagram, DFD	67
Entity-Relationship Diagram, ERD	67
State-transition Diagram, STD	68
Swimlane diagram	70
Descision tree	70
System context diagram	70
Карты диалоговых окон	70
Таблица и древо решений	71
Таблицы событий и реакций	71

УРОВНИ И ТИПЫ ТРЕБОВАНИЙ



Бизнес-уровень

БИЗНЕС-ТРЕБОВАНИЯ

Business requirements – описывают, почему организации нужна такая система, т.е. цели, которые организация должна достичь с помощью ПО.

Записываются в виде документа о концепции и границах (vision and scope document) и в виде:

- устава проекта (project charter).
- вариант использования (business case).
- документа рыночных требований (market requirement document).

БИЗНЕС ПРАВИЛА

Business rules – политики компании, правительственные законы, отраслевые стандарты и вычислительные алгоритмы.

- Отправная точка для бизнес, функциональных, пользовательских требований и атрибутов качества.

Пользовательский уровень

ПОЛЬЗОВАТЕЛЬСКИЕ ТРЕБОВАНИЯ

User requirements – описывают цели и задачи, которые пользователи должны иметь возможность выполнить с помощью ПО. Что пользователь должен иметь возможность делать с системой.

Записываются в виде:

- UseCase диаграмм.
- пользовательских историй.

АТРИБУТЫ КАЧЕСТВА (НЕФУНКЦ.)

Quality attributes описывают различные характеристики системы, которые важны для пользователей, разработчиков и тех, кто будет поддерживать систему дальше (например, скорость вычислений). Описывают «как хорошо» система что-либо делает.

- Записываются в спецификации требований к ПО (software requirements specification, SRS).
- Отправная точка для функциональных требований.

Функциональный уровень

ФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ

Functional requirements определяют, каким должно быть поведение системы в том или ином случае. Что разработчики должны сделать, чтобы пользователи смогли выполнить свои задачи (пользовательские требования) в рамках бизнес-требований.

Записываются в виде:

- спецификации требований к ПО (software requirements specification, SRS) – ТЗ.

СИСТЕМНЫЕ ТРЕБОВАНИЯ

System requirements – описывают требования к продукту, который содержит в себе несколько компонент или подсистем. Сюда входят описания отдельных компонент и интерфейсы взаимодействия между ними (например, касса в магазине).

- Отправная точка для функциональных требований.

ВНЕШНИЕ ИНТЕРФЕЙСЫ (НЕФУНКЦ.)

Взаимодействие системы с внешним миром (подключения к другим системам, аппаратам, пользователям).

- Записываются в спецификации требований к ПО (software requirements specification, SRS).
- Отправная точка для функциональных требований.

ОГРАНИЧЕНИЯ (НЕФУНКЦ.)

Ограничения, которые должна выполнять система (нефункциональные требования).

- Записываются в спецификации требований к ПО (software requirements specification, SRS).

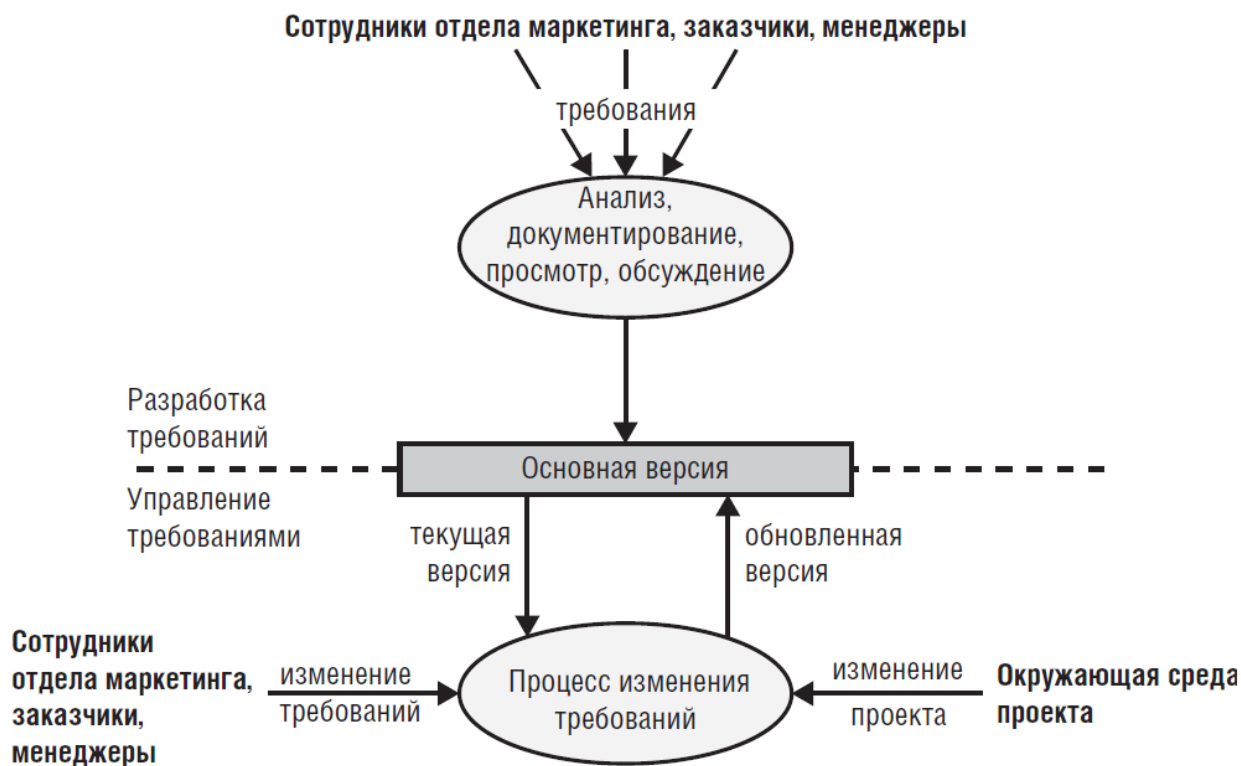
ОБЩИЕ СВЕДЕНИЯ

Требования к проекту

Требования к проекту отличаются от требований к продукту. К требованиям к проекту относятся:

- физические ресурсы, необходимые для разработки.
- потребность в обучении персонала.
- пользовательская документация.
- документация для поддержки системы.
- требования к инфраструктурным изменениям, которые нужны внести.
- требования по правовой защите ПО (патент и т.д.).
- требования по сертификации ПО.
- и т.д.

Разработка и управление требованиями



Разработка требований

1. ВЫЯВЛЕНИЕ

- Определение классов пользователей продукта и других заинтересованных лиц.
- Понимание бизнес-цели.
- Изучение среды, в которой будет использоваться продукт.
- Работа с отдельными людьми, будущими пользователями.

Здесь важно, чтобы требования ориентировались в большей степени на пользователя и его потребности, нежели на сам продукт. В противном случае появляется риск осуществить функции, которые не будут нужны пользователям. При этом аналитик может предлагать и новые функции, о которых пользователи раньше не задумывались.

2. АНАЛИЗ

- выявление пользовательских требований и их отделение от других (функциональных, нефункциональных и т.д.).
- детализация высокоуровневых требований.
- выведение функциональных требований из других типов требований.
- понимание относительной важности атрибутов качества.
- распределение функциональности на компоненты системы или составного ПО.
- согласование приоритетов реализации.
- выявление пробелов в требовании либо излишних требований.

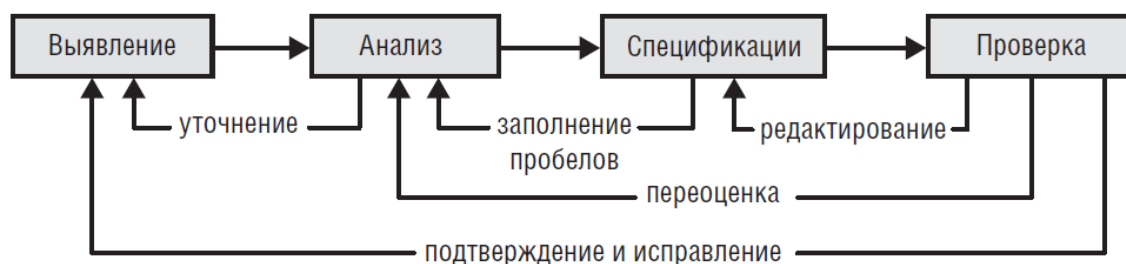
3. ДОКУМЕНТИРОВАНИЕ

- преобразование собранных требований в письменные диаграммы и требования, пригодные для понимания, анализа и использования целевой аудитории.

4. УТВЕРЖДЕНИЕ

- проверка всех требований для устранения недостатков до принятия решения.
- разработка приемочных тестов, с помощью которых можно было бы подтвердить, что продукт удовлетворяет требованиям и поставленным бизнес-целям.

Важно не торопиться при формировании требований. **Ключ к успеху – итеративный подход.**



Управление требованиями

- всегда должен быть определен моментальный снимок требований, который является актуальным на данном этапе разработки или итерации.
- обновление планов проекта и внедрение изменений управляемым образом.
- обсуждение новых обязательств, основанном на оцененном влиянии изменения требования.
- определение отношений и зависимостей между требованиями.
- отслеживание отдельных требований до их проектирования и исходного кода.
- отслеживание состояний требований и действий по их изменению.

Достижение соглашения

Необходимо тесное сотрудничество и взаимодействие разработчиков и клиента (см. билль о правах и обязанностях клиента ПО). Лучше сразу обозначить в той или иной форме права и обязанности каждой заинтересованной стороны.

- Клиенты: требования удовлетворяют их потребностям.
- Разработчики: понимают требования и могут их реализовать.
- Тестировщики: подтверждают, что требования поддаются проверке.
- Руководство: требование соответствует бизнес-целям.

Не следует чрезмерно усердствовать с собранием подписей.

Гораздо важнее создание базового варианта требований

В Agile разработке требования собираются в виде пользовательских историй, и владелец продукта публично принимает или отвергает их на очередной итерации.

Приемы и методы разработки и управления требованиями

СВЯЗЬ С МЕТОДОЛОГИЯМИ РАЗРАБОТКИ

- Водопад (waterfall) – практически все требования определяются в самом начале.
- Итеративный (RUP, Rational Unified Process) – основной объем требований определяется на первой итерации.
- Гибкий (agile) – требования формулируются примерно в равном объеме на каждой итерации.

ПРИЕМЫ РАЗРАБОТКИ ТРЕБОВАНИЙ

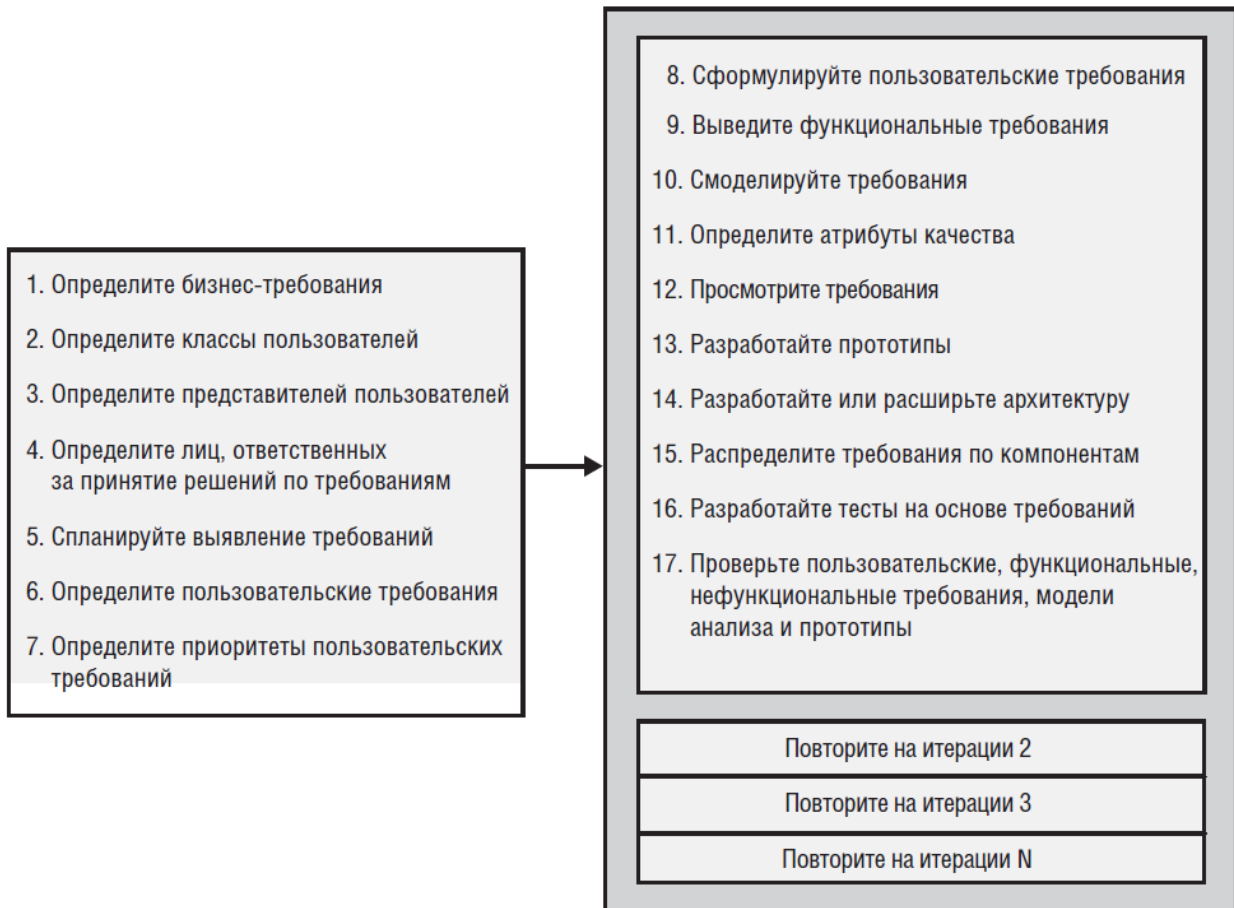
Сбор информации	Анализ	Спецификации	Проверка
• Определите концепцию продукта и границы проекта • Определите классы пользователей • Выделите из пользователей ярких сторонников продукта • Создайте фокус-группы • Определите пользовательские требования • Определите системные события и реакцию на них • Проведите интервью для выявления требований • Проведите семинары по выявлению требований • Наблюдайте за пользователями на рабочих местах • Раздайте опросные листы • Выполните анализ документов • Изучите отчеты о проблемах • Повторно задействуйте существующие требования	• Смоделируйте среду приложения • Создайте прототипы • Проанализируйте осуществимость • Расставьте приоритеты для требований • Создайте словарь данных • Смоделируйте требования • Проанализируйте интерфейсы • Распределите требования по подсистемам	• Используйте шаблон спецификации требований • Определите источники требований • Задайте каждому требованию уникальный идентификатор • ЗадOCUMENTИРУЙТЕ бизнес-правила • Определите атрибуты качества	• Изучите документы с требованиями • Протестируйте требования • Определите критерии приемлемости • Смоделируйте требования

ПРИЕМЫ УПРАВЛЕНИЯ ТРЕБОВАНИЯМИ

Управление требованиями	Обучение	Управление проектом
• Определите процесс управления изменениями • Проанализируйте, какое влияние оказывают изменения • Определите базовую и контрольную версии наборов требований • Отслеживайте хронологию изменений • Отслеживайте состояние требований • Отслеживайте проблемы с требованиями • Создайте матрицу связей требований • Используйте средство управления требованиями	• Обучите аналитиков требований • Ознакомьте представителей пользователей и менеджеров с требованиями • Обучите разработчиков основам предметной области • Определите процесс разработки требований • Создайте словарь терминов	• Выберите соответствующий цикл разработки проекта • Планируйте подход к работе с требованиями • Оцените объем работ по реализации требований • Планируйте на основании требований • Определите лиц, ответственных за принятие решений по требованиям • Своевременно пересматривайте обязательства • Управляйте рисками, касающимися требований • Отслеживайте объем работ по реализации требований • Делайте выводы из полученного опыта

РАЗРАБОТКА ТРЕБОВАНИЙ

Общий план разработки требований



Определение бизнес-цели

ОБЩИЕ СВЕДЕНИЯ

- Концепция продукта остается неизменной на протяжении всех итераций.
- Границы определяются каждый раз по-новому.
- Концепция должна решать проблему.
- Бизнес-интересы заинтересованных лиц могут противоречить друг-другу (пример, продажи).
- Не упускать границы проекта! Всегда опираться на бизнес-цели.
- Оценка эффекта от изменения границ проекта. Можно лавировать в каких-либо измерениях, которые являются степенями свободы в данном проекте.

ШАБЛОН ДОКУМЕНТА О КОНЦЕПЦИИ И ГРАНИЦАХ

1. Бизнес-требования
 - а. Исходные данные (предпосылки к созданию продукта).

- b. Возможности бизнеса (рыночные возможности – для коммерческого ПО, решаемые бизнес-задачи – для корпоративных систем; сравнительная характеристика ПО по сравнению с уже существующими; задачи которые решает ПО; краткое описание других технологий и ресурсов, необходимых для решения проблемы).
 - c. Бизнес-цели (конкретные цели: увеличить продажи на 15% в первый год пользования и т.д.).
 - d. Критерии успеха (решение бизнес-цели можно решить только в конце проекта, критерии – для отслеживания прогресса, частичные бизнес-цели).
 - e. Положении о концепции (для кого, особенности ПО, имя, категория, функции, ключевые преимущества, причина для покупки, отличие от других продуктов или текущих систем и процессов).
 - f. Бизнес-риски (связанные с разработкой или не разработкой этого продукта; отличаются от рисков проекта).
 - g. Предположения и зависимости (задокументировать предположения всех заинтересованных лиц).
2. Рамки и границы
- a. Основные функции (основные функции, включая отличие от предыдущей версии, конкурирующего ПО и т.д.).
 - b. Объем первоначально запланированной версии.
 - c. Объем последующих версий.
 - d. Ограничения и исключения (описать то, чего не будет в определенной версии).
3. Бизнес-контекст
- a. Профили заинтересованных лиц (что принесет конкретному пользователю или классу пользователей).
 - b. Приоритеты проекта:
 - i. Пять измерений: функции (объем), качество, график, затраты, кадры.
 - ii. Три категории: ограничения, ведущий фактор (нельзя менять), степень свободы (можно варьировать).
 - c. Особенности развертывания (действия и средства, необходимые для развертывания).

СПОСОБЫ ПРЕДСТАВЛЕНИЯ ГРАНИЦ ПРОЕКТА

Границы определяются для конкретной итерации, в отличие от концепции (см. примеры в Вигерс).

- Контекстная диаграмма (оконечные элементы, расположенные вне системы).
- Карта экосистемы (взаимодействие с другими системами).
- Древо функций (описание функций).
- Список событий.

Определение классов пользователей

КЛАССИФИКАЦИЯ ПОЛЬЗОВАТЕЛЕЙ

Группировать можно по различным критериям:

- по привилегиям (гость, обычный пользователь, админ).
- по задачам, которые решают пользователи.
- по используемым функциям.

- по частоте использования продукта.
- по родному языку.
- по виду доступа к системе (прямой или косвенный).

Одна из общих классификаций:

- Привилегированные – те, для кого делается ПО.
- Непривилегированные – те, кто работать с ПО по каким-либо причинам не должны (например, интернет-боты в случае сайта).
- Игнорируемые – те, кто получают такой продукт, какой есть. Их мнение не важно.

Нельзя упускать косвенных пользователей. Например, те, кто будут работать с отчетами, генерируемыми программой.

ОПРЕДЕЛЕНИЕ КЛАССОВ

- Придумать как можно больше возможных классов пользователей.
- Опирается на структурную схему компании, так как практически все классы пользователей находятся в ней.
- Выявить группы с похожими потребностями и объединить их.
- Общее число классов не должно быть больше 15.

Для каждого класса важно определить в виде документа:

- Имя класса и его тип (привилегированный или нет).
- Примерное количество представителей класса.
- Описание каждого класса (функции, которые нужны и т.д.).
- Архетип пользователя (типичный представитель класса – может быть реальным, может быть виртуальным).

Важно! Приоритеты требований зависят от класса пользователей. Также важно учитывать опыт и квалификацию пользователей.

ПРЕДСТАВИТЕЛИ ПОЛЬЗОВАТЕЛЕЙ

При разработке любого типа ПО важно привлечь представителей пользователей всех классов, которые будут выражать мнение большого количества пользователей.

Важно уважать добрую волю этого человека (или группы лиц), но при этом избегать возможных ошибок, связанных с субъективностью мнения.

Обязательно присутствие в фокус-группе представителей всех классов.

СТОРОННИК ПРОДУКТА

Это пользователи, которые помогают формулировать требования.

Каждый сторонник продукта – посредник, между классом пользователей и аналитиком.

Сторонник продукта должен пользоваться авторитетом в компании (для которой разрабатывается ПО).

Здесь важно помнить, чтобы мнение сторонника сотрудника было максимально объективным.

Можно пригласить и со стороны, например, текущего специалиста в той области, для которой разрабатывается ПО.

У каждого сторонника продукта должны быть определены обязанности (стр. 127):

Категория	Действия
Планирование	Уточнение границ и ограничений продукта Определение интерфейсов для работы с другими системами Оценка влияния новой системы на бизнес-операции Разработка путей перехода со старых приложений или от ручных операций Определение применимых стандартов и сертификационных требований
Требования	Сбор требований от других пользователей Разработка сценариев и вариантов использования продукта, а также пользовательских историй Разрешение конфликтов между предложенными требованиями в рамках классов пользователей Определение приоритетов реализации Определение атрибутов качества и производительности Оценка прототипов Сотрудничество с ответственными за принятие решений над разрешением противоречий требований, поступивших от разных заинтересованных лиц Предоставление специализированных алгоритмов
Проверка и утверждение	Рецензирование документов с требованиями Определите критериев приемлемости Разработка приемочных тестов на основе сценариев использования Создание наборов тестовых данных на основе бизнес-данных Выполнение бета-тестирования и приемочных испытаний пользователями
Помощь пользователям	Написание фрагментов руководств пользователя и справочных систем Подготовка части материалов для учебных пособий Демонстрация системы коллегам
Управление изменениями	Оценка исправлений дефектов и предложений об усовершенствовании системы и определение порядка их реализации Динамическая корректировка границ будущих выпусков Оценка того, как предполагаемые изменения требований повлияют на пользователей и на бизнес-процессы Участие в принятии решений о внесении изменений

Возможные ловушки:

- Менеджеры в последний момент меняют мнение сторонника продукта.
- Сторонник продукта учитывает только свои потребности, забыв о своих коллегах.
- Соглашается во всем с аналитиком (бесполезен).
- Пытается выдвигаться от класса пользователей, к которому не принадлежит.

В гибкой методологии разработки владелец продукта является представителем пользователя.

РАЗРЕШЕНИЕ ПРОТИВОРЕЧИЙ

Разногласия между...	Как разрешать
...отдельными пользователями	Решение принимает сторонник или владелец продукта
...классами пользователей	Предпочтение отдается наиболее важному классу пользователей
...сегментами рынка	Предпочтение отдается сегменту, оказывающему наибольшее влияние на успех бизнеса
...корпоративными клиентами	Решение определяется на основе бизнес-целей
...пользователями и менеджерами пользователей	Решение принимает сторонник или владелец продукта класса пользователей
...разработчиками и клиентами	Предпочтение отдается клиентам, но с учетом бизнес-целей
...разработчиками и специалистами по маркетингу	Предпочтение отдается специалистам по маркетингу

Выявление требований

МЕТОДЫ ВЫЯВЛЕНИЯ ТРЕБОВАНИЙ

	Интервью	Семинары	Фокус-группы	Наблюдение	Опросные листы	Анализ системных интерфейсов	Анализ пользовательского интерфейса	Анализ документов
ПО для массового рынка	X		X		X			
Внутрикорпоративное ПО	X	X	X	X		X		X
Замена существующей системы	X	X		X		X	X	X
Обновление существующей системы	X	X				X	X	X
Новое приложение	X	X				X		
Реализация коробочного ПО	X	X		X		X		X
Встроенные системы	X	X				X		X
Географически распределенные заинтересованные лица	X	X			X			

1. Интервью

Подходит для всех, в том числе и для топ-менеджеров, которые заняты.

Ключи к успеху: установка контакта, соблюдение рамок и границ проекта, предложение своих идей, активное слушание, заранее подготовленные вопросы и модели.

2. Семинары

Могу входить разработчик, тестер, клиент, архитектор и т.д.

Ключи к успеху: определить и отслеживать правила (время, этикет), обеспечить все роли в команде (ведущий, секретарь), планирование повестки дня заранее, придерживаться границ проекта,

фиксировать темы для дальнейших обсуждений, ограничивать некоторые дискуссии по времени, тщательный отбор команды до 5 человек, вовлекать в обсуждение каждого.

3. Фокус-группы

Представительная группа пользователей (см. выше). Главное здесь – правильно руководить встречами.

4. Наблюдение

Иногда можно упустить много деталей, кажущиеся пользователям очевидными. Наблюдения за пользователями позволяют рассмотреть эти детали. Два типа: активное и пассивное.

5. Опросные листы

Подходят для большого количества пользователей. Главное – правильно создать и протестировать опрос. Желательно проконсультироваться со специалистом.

6. Анализ системных интерфейсов

Некоторые требования (интерфейсов) можно получить из анализа систем, с которым должно взаимодействовать ПО. Например, из контекстных диаграмм и эокарт (см. выше).

7. Анализ пользовательского интерфейса

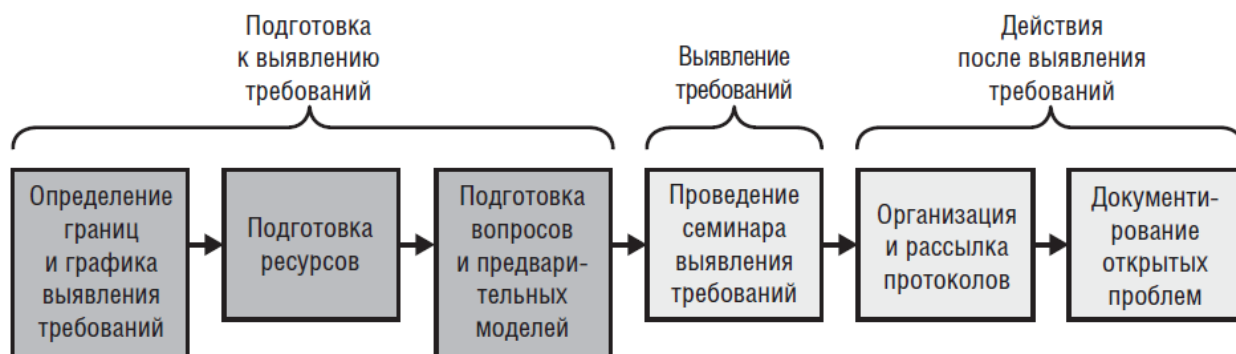
Можно использовать на примере существующего ПО, которое следует изменить или заменить.

8. Анализ документов

Руководство пользователя, спецификация требований, бизнес-процессы, корпоративные и отраслевые стандарты, отчеты о проблемах и запросы пользователей.

Главный риск – устаревание документов.

ПЛАНИРОВАНИЕ ВЫЯВЛЕНИЯ ТРЕБОВАНИЙ



- Определить цели выявления требований.
- Решить, какие приемы будут применяться в разных группах.
- Определить график и ресурсы (помещения, доски и т.д.).
- Документы и системы, необходимые для независимого анализа (анализ документов, интерфейса и систем).
- Определить ожидаемые результаты выявления требований (создать варианты использования, окончательную спецификацию и т.д.).
- Определить риски, связанные с выявлением требований.

ПОДГОТОВКА К ВЫЯВЛЕНИЮ ТРЕБОВАНИЙ

- Определение границ и графика выявления требований.

- Подготовка ресурсов (переговорные, время участников и т.д.).
- Сбор информации о заинтересованных лицах.
- Подготовка перечня вопросов (открытые вопросы, чтобы понять бизнес-процессы пользователя. Например, что спрашивал бы новичок?).
- Заранее подготовить предварительные модели (варианты использования, потоки процессов).

ВЫЯВЛЕНИЕ ТРЕБОВАНИЙ

- Объяснение своих методов участникам.
- Качественное протоколирование.

ПОСЛЕ ВЫЯВЛЕНИЯ ТРЕБОВАНИЙ

- Организация и рассылка протоколов.
- Документирование открытых проблем.

КЛАССИФИКАЦИЯ ТРЕБОВАНИЙ

Каждое требование так или иначе необходимо отнести к типу требований (см. пример стр 156):

- Бизнес-требование (финансовые, рыночные и другие коммерческие вопросы).
- Пользовательские требования (варианты использования, что может делать пользователь).
- Бизнес-правила (соответствие чему-либо, кто-то что-то должен или не должен).
- Функциональные требования (ожидаемое поведение системы в определенных условиях).
- Атрибуты качества (как хорошо что-либо делается).
- Требования к внешнему интерфейсу (откуда, что получать, в каком формате отправлять).
- Ограничения (размеры файлов, тип шифрования и т.д.).
- Определение данных (форматы данных, типы, значения по умолчанию).
- Идеи, касающиеся решений (когда предлагается не требование, а решение – тут главное понять, насколько оно справедливо).



ОКОНЧАНИЕ СБОРА ТРЕБОВАНИЙ

Основные маркеры, что пора завершать сбор требований:

- Пользователя начинают говорить о функциях, которые можно было бы реализовать в будущих версиях.
- Разработчики и тестеры не задают вопросов.
- Новые требования имеют низкий приоритет.
- Повторное описание уже обсуждавшийся проблемы.
- Предлагаемые функции и требования выходят за рамки проекта.

ПОИСК УПУЩЕННЫХ ТРЕБОВАНИЙ

- Раскладывать и детализировать требования высокого уровня.
- Каждый класс пользователей должен предоставить информацию, у каждого требования должен быть соответствующий класс пользователей.
- Подробная документация: на каких функциональных требованиях основаны требования к системе, пользовательские требования и т.д. Это позволит удостовериться, что описана вся необходимая функциональность (т.е. тут проверка в обратную сторону).
- Проверка пограничных и предельных значений (особенно в числах).
- Использовать разнообразные формы представления о требованиях.
- Наборы со сложной булевой логикой часто оказываются неполными. Использовать таблицы и деревья решений.
- Сверять контрольный список (который создал сам) с уже определенными функциями на предмет выявления пробелов.
- Создание модели данных. Все сущности данных должны иметь соответствующую функциональность (создание, чтение, изменение, удаление).

Остерегаться паралича разработки требований: не надо тратить много времени на выявление требований, боясь упустить какое-либо из них.

ДОПОЛНИТЕЛЬНО

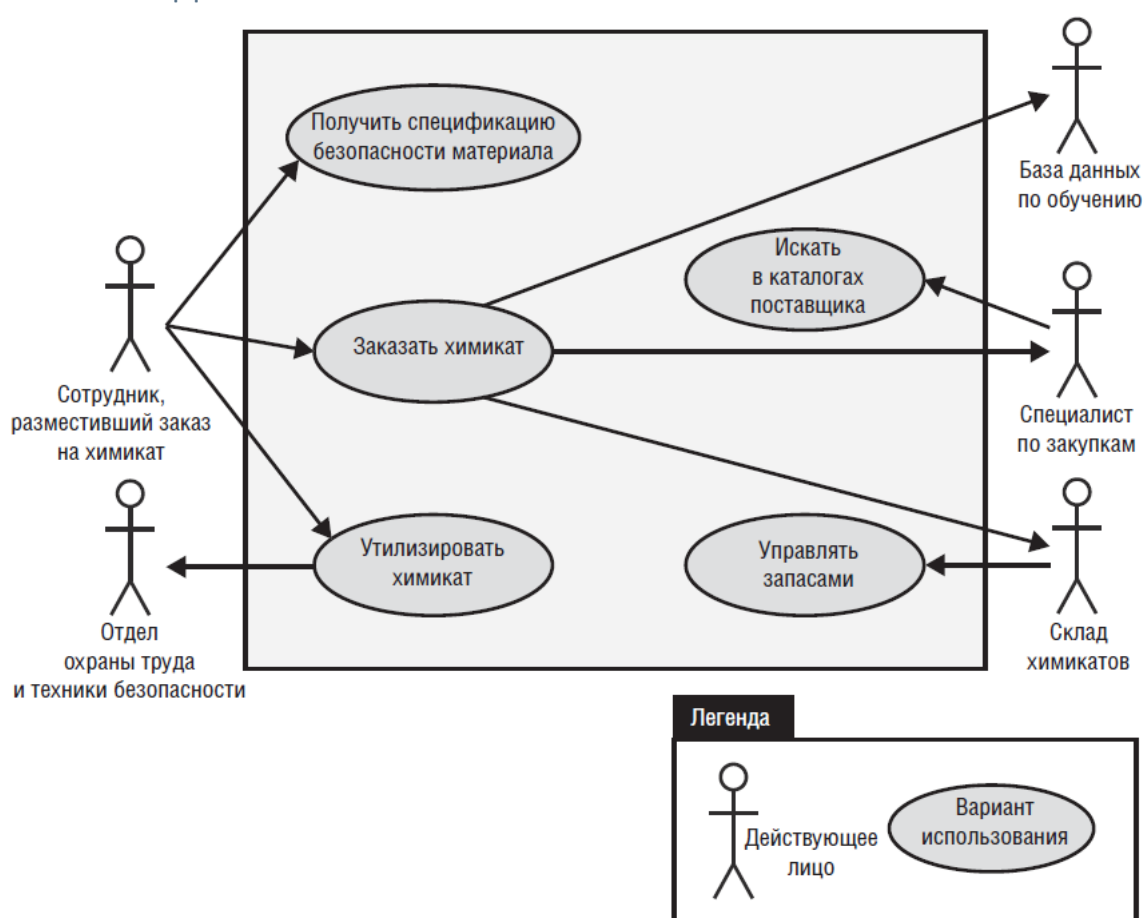
- Нужно правильно определять границы проекта – они должны быть не слишком узкими, и не слишком широкими. В первом случае – пользовательские требования выходят за рамки проекта, во втором – необходимо дополнить требования, чтобы принести бизнес-пользу.
- Стараться обозначить подразумеваемые требования (то, что люди ожидают получить, явно не выражая) и неявные требования (то, что необходимо по причине другого требования, но явно не сформулированы). Тут надо выявлять то, что пользователи считают само-собой разумеющимся.

Варианты использования

ОБЩЕЕ

- Используется для перехода от пользовательских требований к функциональным требованиям.
- Описывает взаимодействие действующих лиц (абстрактные лица, не классы пользователей) с системой.
- Основной инструмент: UML UseCase Diagram.
- Каждый вариант использования потом расписывается более подробно.

ПРИМЕР ДИАГРАММЫ UML



Есть также расширения и включения. Расширения – расширяет основные направления, например, искать в каталогах поставщика. Включения – вариант с общей функциональностью для других вариантов.

ОПИСАНИЕ ВАРИАНТА ИСПОЛЬЗОВАНИЯ

Основные параметры, которые следует описать (см. рис. ниже):

- Уникальный идентификатор и имя цели варианта использования.
- Автор варианта и дата создания.
- Краткое описание цели варианта использования.
- Триггер
- Предварительные условия – что должно быть выполнено в системе, до выполнения варианта действия.
- Выходные условия – состояние системы после успешного выполнения действия (внутренние изменения системы, физические результаты, вывод пользователю информации).
- Нормальное направление развития варианта – наиболее частый вариант использования.
- Альтернативное направление развития варианта – другие успешные варианты использования.
- Исключения – условия, препятствующие успешному выполнению действия. Обязательно нужно описать исключения, которые не позволяют реализовать успешно вариант использования.
- Приоритет – высокий, средний, низкий.
- Частота использования – кто, как часто использует.

- Бизнес правила – например, скидки при покупке на определенную сумму и т.д.
- Специальные требования.
- Предположения.

Не всегда имеет смысл документировать все подробно. Здесь нужно с умом подходить к вопросу.

Идентификатор и название:	UC-4 Запросить химикат	
Автор:	Лори	Дата создания: 22.08.13
Основное действующее лицо:	Сотрудник, разместивший заказ на химикат	Дополнительное действующее лицо: Покупатель; Склад химикатов; База данных по обучению
Описание:	Сотрудник, разместивший заказ на химикат, указывает в запросе необходимый химикат, вводя его название или идентификатор или импортируя его структуру из соответствующего графического средства. Система выполняет запрос, предлагая контейнер с химикатом со склада или позволяя создать запрос на заказ у поставщика.	
Триггер:	Сотрудник указывает, что хочет заказать химикат.	
Предварительные условия:	PRE-1. Личность пользователя аутентифицирована. PRE-2. Пользователь имеет право запрашивать химикаты. PRE-3. База данных по запасам химикатов в данный момент доступна.	
Выходные условия:	POST-1. Запрос сохраняется в Chemical Tracking System. POST-2. Запрос отправлен на склад химикатов или поставщику.	
Нормальное направление развития варианта использования:	4.0 Запросить химикат со склада 1. Сотрудник указывает требуемый химикат. 2. Система перечисляет контейнеры с необходимым химикатом, имеющиеся на складе. 3. Сотрудник может просмотреть историю любого контейнера. 4. Сотрудник выбирает определенный контейнер или просит отправить запрос поставщику (см. 4.1). 5. Сотрудник вводит остальную информацию, чтобы завершить запрос. 6. Система сохраняет запрос и отправляет его на склад химикатов.	
Альтернативное направление развития варианта использования:	4.1 Запросить химикат у поставщика 1. Сотрудник ищет химикат по каталогам поставщика (см. 4.1.E1). 2. Система отображает список поставщиков, где также указаны размеры, класс и цена контейнеров. 3. Сотрудник выбирает поставщика, размер, класс и количество контейнеров. 4. Сотрудник вводит остальную информацию, необходимую для запроса. 5. Система сохраняет запрос и перенаправляет его поставщику.	
Исключения:	4.1.E1. Химиката нет в продаже 1. Система отображает сообщение «У поставщиков нет такого химиката». 2. Система предлагает сотруднику запросить другой химикат или выйти из программы. 3а. Сотрудник просит запросить другой химикат. 4а. Система заново начинает нормальное направление варианта использования. 3б. Сотрудник решает выйти из системы. 4б. Система завершает вариант использования.	
Приоритет:	Высокий	
Частота использования:	Примерно пять раз в неделю каждым химиком, 200 раз в неделю каждым работником склада.	
Бизнес-правила:	BR-28, BR-31	
Специальные требования:	Система должна импортировать химические структуры в стандартной закодированной форме из любых средств, поддерживающих рисование химических структур.	
Предположения:	Импортированные химические структуры должны быть верными.	

МАКРОВАРИАНТЫ

Могут также использоваться варианты использования, которые включают в себя несколько вариантов. Например, покупка товара в интернет магазине (поиск, положить в корзину, оплатить). При этом каждый предыдущий вариант должен оставлять систему в состоянии, соотв. предварительным условиям для последующего варианта (согласование входных и выходных условий).

МЕТОДИКИ ОПРЕДЕЛЕНИЯ ВАРИАНТОВ ИСПОЛЬЗОВАНИЯ

- сначала определить действующих лиц, затем бизнес-процессы, поддерживаемые системой, и варианты использования для действий, в которых участвуют действующие лица и система;
- выразить бизнес-процессы в терминах определенных сценариев, обобщить сценарии в варианты использования и определить действующих лиц для каждого варианта;
- используя описание бизнес-процесса, задать вопрос: «Какие задачи должна система выполнить, чтобы реализовать этот процесс или преобразовать входные данные в выходные?» Эти задачи могут быть вариантами использования;
- определить внешние события, на которые система должна реагировать, а затем соотнести эти события с участвующими лицами и конкретными вариантами использования;
- применить CRUD-анализ (Create, Read, Update, Delete — создание, чтение, обновление, удаление) для определения сущностей данных, которым нужны варианты использования для создания, чтения, обновления, удаления или других операций с данными (см. главу 13);
- проанализировать контекстную диаграмму, задавшись вопросом: «Какие цели нужно каждой из внешних сущностей достичь с использованием системы?»

Не надо включать все требования в вариант использования. Здесь необходимо придерживаться границ проекта. Более того, важно правильно расставлять приоритеты. Например, если вариант использования имеет высокий приоритет, то связанные с ним функциональные требования также имеют высокий приоритет.

Также надо всегда проверить корректность составленных вариантов использования.

ПЕРЕХОД К ФУНКЦИОНАЛЬНЫМ ТРЕБОВАНИЯМ

Варианты использования не описывают, что делает система, если предварительные требования не выполняются – это задача аналитика.

Вообще, задача аналитика, перевести пользовательские требования в функциональные, чтобы разработчики смогли начать работать.

Способы документации функциональных требований, связанных с вариантом использования.

1. Только варианты использования – функциональные требования описываются внутри каждого варианта использования. Повторные ф.т. можно определять ссылками.
2. Варианты использования и функциональные требования – важно отслеживать изменения. Рекомендуется использовать средства управления требованиями.
3. Только функциональные требования – упорядочить функциональные требования по вариантам использования. Документ о пользовательских требованиях может не составляться.
4. Варианты использования и тесты – полные спецификации требований и приемочные тесты.

ПОДВОДНЫЕ КАМНИ

- Слишком много вариантов использования.

- Очень сложные варианты использования (но нет смысла дробить большие, но верные варианты использования).
- Включение дизайна в варианты использования. Дизайн не должен диктовать направление разработки требований.
- Включение определения данных. Данные нужно собирать в отдельном словаре данных.
- Варианты использования, непонятные пользователям. Должны быть простыми и эффективными.

ПРИОРИТЕТЫ

Высший приоритет отдается вариантам использования, если они:

- являются основными бизнес-процессами.
- используются многими пользователями.
- запрошены привилегированными классами пользователей.
- предоставляют возможности, нужные для работы с регулирующими органами.
- функции других системы зависят от их наличия.

Бизнес-правила

КЛАССИФИКАЦИЯ БИЗНЕС-ПРАВИЛ

1. Факты

Это верные утверждение о бизнесе на определенный момент. Пример: на каждый химический контейнер нанесен уникальный штрих-код.

2. Ограничения

Определяют, какие операции система или пользователи не могут выполнять.

1. Политики организации (пример: договор о найме человека младше 18 лет должен подписывать заведующий лабораторией).
Если ограничения связаны с доступом лиц, удобно использовать **матрицы ролей и разрешений**.
2. Государственные нормативы (пример: экипажи авиарейсов должны отдыхать не менее 8 часов в сутки).
3. Отраслевые стандарты (пример: веб-приложения не должны содержать HTML-тегов, не соответствующих стандарту HTML-5).

3. Активаторы операций

Правило, активирующее определенное действие при выполнении определенных условий.

Пример: если срок хранения контейнера с химикатом истек, нужно уведомить лицо, у которого находится контейнер.

4. Выводы

Создает новый факт на основе других фактов.

Пример: если платеж не поступил в течение 30 календарных дней с момента отправки счета, то счет считается просроченным.

5. Вычисления

Преобразует существующие данные в новую информацию с использованием формул и алгоритмов.

Пример: формирование цены в зависимости от массы заказа и пункта назначения.

Пример: предоставление скидки (важно, чтобы интервалы не пересекались, 1-10, 11-20, а не 1-10, 10-20).

6. Атомарные бизнес-правила

Правила, которые нельзя разделить на более мелкие.

Пример: при продлении аренды срок сдачи увеличивается на 3 дня.

ДОКУМЕНТИРОВАНИЯ БИЗНЕС-ПРАВИЛ

Отлично, если они уже есть в организации – например, в виде документа.

Организации следует иметь набор таких бизнес-правил, которые использовались бы во многих проектах.

Если такого документа нет, то в рамках проекта необходимо создать каталог бизнес-правил, на который потом будут ссылаться другие требования. При этом нужно соблюдать правила:

- Бизнес-правила должны быть сформулированы в простой форме.
- Для каждого бизнес-правила должны быть определены:
 - Уникальный идентификатор
 - Определение правила
 - Тип правила (см. выше)
 - Статичное или динамическое
 - Источник
 - Дата занесения в каталог

Можно создать свой шаблон для бизнес-правил, который можно будет неоднократно использовать в различных проектах.

ВЫЯВЛЕНИЕ БИЗНЕС-ПРАВИЛ

Стандартные места и способы поиска бизнес-правил:

- Общеизвестные знания организации – можно собрать у людей, давно работающих в компании.
- Унаследованные системы, в которых реализованы бизнес-правила (путем обратного инжиниринга).
- Моделирование бизнес-процессов, что требует от аналитика искать правила.
- Анализ имеющейся документации (предыдущие проекта, нормативные документы, отраслевые стандарты).
- Анализ данных и их возможных состояний, условий, в которых пользователь или системное событие может изменить состояние объекта данных.
- Отделы нормативно-правового обеспечения компании.

Важно проверять актуальность бизнес-правил. Более того, не надо искать все существующие бизнес-правила. Нужны только те, которые находятся в пределах границы проекта.

Документирование требований

ШАБЛОН SRS

1. Введение
 - 1.1. Назначение (номер выпуска продукта, название, к какой части системы относится и т.д.)
 - 1.2. Соглашения, принятые в документе (стандарты и типографические соглашения)
 - 1.3. Граница проекта (краткое описание ПО и его назначения, связь с корпоративными целями, ссылка на документ о концепции и границах)
 - 1.4. Ссылки (другие ресурсы, стандарты и так далее)
2. Общее описание
 - 2.1. Общий взгляд на продукт (контекст и происхождение продукта, это новая система или замена старой, соотношение с другими системами – модели эконокарта или контекстная диаграмма IDEF0)
 - 2.2. Классы и характеристики пользователей (описание классов и характеристик пользователей)
 - 2.3. Операционная среда (рабочая среда, в которой будет работать система; другие ПО, с которыми будет взаимодействовать система)
 - 2.4. Ограничения дизайна и реализации (опционально: например, язык программирования; часто не всегда оправданы требования и по дизайну – тут надо следить)
 - 2.5. Предположения и зависимости (например, нужно установить Net Framework 4.5 или выше).
3. Функции системы
 - 3.1. Функция системы X
 - 3.1.1. Описание X (краткое описание функции, ее приоритет)
 - 3.1.2. Функциональные требования (по пунктам, все функциональные требования, связанные с данной функцией; реакция продукта на ожидаемые ошибки; функциональным требованиям можно сопоставить атрибуты, например, основание, источник, состояние)
4. Требования к данным
 - 4.1. Логическая модель данных (визуальное представление объектов и наборов данных, которые обрабатывает система, а также отношения между этими данными; используется ERD и UML)
 - 4.2. Словарь данных (определяет структуру данных, их тип, длину, формат и т.д.)
 - 4.3. Отчеты (если система будет генерировать отчеты, то тут их характеристики; логическое описание и желательно пример)
 - 4.4. Получение, целостность, хранение и утилизация данных (если важно, описать, как система получает и обрабатывает данные; требования к целостности данных; как система должна поступать с локальными копиями, промежуточными архивами и т.д.)
5. Требования к внешним интерфейсам
 - 5.1. Пользовательские интерфейсы (логические характеристики каждого интерфейса, стандарты отображения чего-либо, сочетания клавиш, специальные возможности для слабовидящих и т.д.)
 - 5.2. Интерфейсы ПО (связи системы с другими компонентами ПО, в т.ч. БД, другие приложения, библиотеки и т.д.; описание сообщений между, данных, обмен которыми происходит; службы, нужные для внешних компонентов ПО; меры и ограничения безопасности)
 - 5.3. Интерфейсы оборудования (характеристики каждого интерфейса между компонентами ПО и оборудования системы, если такие есть; типы поддерживаемых устройств, элементы управления ПО; входные и выходные данные, а также их форматы)
 - 5.4. Коммуникационные интерфейсы (электронная почта, браузер, сетевые протоколы; форматы сообщений; ограничения этих интерфейсов, например, допустимые вложения в почте)
6. Атрибуты качества
 - 6.1. Удобство использования (тут помощь дизайнеру; спец. возможности, требования к удобству использования и т.д.)
 - 6.2. Производительность (конкретные требования к производительности)

- 6.3. Безопасность (защита данных, ограничения доступа и т.д.; как правило, здесь все связано с бизнес-правилами)
 - 6.4. Техника безопасности (возможные потери и ущерб, соответствие сертификатам безопасности)
 - 6.5. [Другие] (возможность установки, доступность и т.д.)
 - 7. Требования к интернационализации и локализации (язык, законы, точки-запяты и прочее)
 - 8. Остальные требования
- Приложение А: Словарь терминов (можно вести отдельно).
- Приложение Б: Модель анализа (можно картинки вставлять походу).

Не надо включать пользовательские интерфейсы в SRS. Максимум – наброски.

ОБЩИЕ ЗАМЕЧАНИЯ

Спецификация требований к ПО нужна различным участникам проекта:

- клиенты, отдел маркетинга и специалисты по продажам – чтобы знать общее представление.
- менеджеры проекта – расчет графиков работы, затрат и ресурсов.
- команда разработчиков ПО – представление о том, что нужно разрабатывать.
- тестировщики – составление тестов, основанных на требованиях.
- специалисты по обслуживанию – представление о функциональности составной части продукта.
- технические писатели – создание руководств пользователя и т.д.
- специалисты по обучению – создание учебных материалов.
- юридический отдел – соответствие продукта стандартам.
- субподрядчики – распределение ответственности.

Лучше всего сделать набор шаблонов для документов в пределах организации. Спецификаций должно быть столько, сколько нужно. И разбивать ее тоже нужно с умом (по подсистемам и т.д., если это целесообразно).

АТРИБУТЫ ХОРОШЕЙ ДОКУМЕНТАЦИИ

- Использование шаблона для SRS.
- Единообразное название разделов и подразделов.
- Использование средств визуального выделения.
- Наличие оглавления.
- Нумерация рисунков, таблиц и т.д.
- Использование гиперссылок.
- Включение графического представления.
- Тщательное редактирование.

НОМЕНКЛАТУРА ТРЕБОВАНИЙ

Возможны следующие варианты нумерации требований:

- **Уникальный порядковый номер**
 - Пример: BR-21. Номер при этом никогда не повторяется. Если требование удалено, нумерация идет дальше по порядку, чтобы не было путаницы.
- Иерархическая нумерация
 - Пример: 1.2.3. Просто и компактно, но бывает очень громоздко.
- **Текстовые теги**
 - Пример: Discount.Error.1

Написание идеальных требований

ОБЩАЯ ЧАСТЬ

Главные принципы создания требований:

- любой, прочитавший требование, должен прийти к той же интерпретации, что и любой другой читатель этого требования.
- интерпретация всех читателей требования должны совпадать с тем, что хотел сказать автор.

Существуют два подхода написания требований:

- системная точка зрения (система должна выполнять ...)
- пользовательская точка зрения (пользователь должен иметь возможность выполнить ...)

ХАРАКТЕРИСТИКИ ИДЕАЛЬНОГО НАБОРА ТРЕБОВАНИЙ

- Полнота (нет ни одного TBD)
- Согласованность (требования не конфликтуют друг с другом)
- Способность к модификации
- Осуществимость (в рамках бюджетных, временных и ресурсных ограничений проекта)
- Отслеживаемость (требования можно отследить вплоть до бизнес-целей)
- Назначение приоритетов (требования должны быть с приоритетом)
- Недвусмысленность (разные люди должны интерпретировать требование одинаково)
- Проверяемость (для тестировщиков – возможность проверить реализацию той или иной функции)

ЯЗЫК И СТИЛЬ

- Лучше использовать одни и те же слова все время.
- Слова должны быть однозначными (использовать «должна», а не «может»; всегда нужно давать количественную оценку, например, какая степень достаточности; не использовать «т.д.», «т.п.»).
- Требования следует писать в положительном ключе (что может сделать система, а не то, что не может).
- Не надо использовать пассивный залог.
- Сложные логические требования должны определять все значения логических исходов (можно оформлять в виде таблиц).

Моделирование требований

Моделирования необходимо, чтобы обеспечить более высокий уровень понимания, чем это дает SRS.

См. Приложение А: Моделирование требований.

Требования к данным

МОДЕЛИРОВАНИЕ ДАННЫХ: DFD И ERD

Для моделирования данных очень удобно использовать:

- Data Flow Diagram (DFD) – диаграмма потока данных,
- Entity-Relationship Diagram (ERP) – диаграмма «сущность-связь».

ERP можно использовать как в нотации Чена, так и в виде Class Diagram UML или IDEF1X.

Описание этих моделей данных в Приложение А: Моделирование требований.

Помимо высокоуровневых моделей необходимо использовать словарь данных, чтобы полностью определить требования к данным.

СЛОВАРЬ ДАННЫХ

Словарь данных (data dictionary) раскрывает суть модели данных, содержит в себе подробную информацию о всех типах данных. Основные поля данных: элемент, описание, структура и тип, длина, значения.

Типы элементов словаря данных:

- Простейшие элементы, которые не могут быть далее разбиты на более мелкие элементы.
- Структура, которая состоит из нескольких простейших элементов, структур.
- Повторяющаяся группа – если в структуре содержится несколько повторяющихся элементов данных, структур. Обозначается в формате минимум: максимум. Пример: «1,10{Химикат}».

Элемент данных	Описание	Структура или тип данных	Длина	Значения
Заказ химиката	Заказ нового химиката со склада или у поставщика	Идентификатор заказа + Сотрудник, разместивший заказ на химикат + Дата заказа + Счет затрат + 1:10{Заказанный химикат}		
Пункт назначения поставки	Место, куда нужно доставить заказанный химикат	Строение + Номер лаборатории + Отделение лаборатории		
Количество контейнеров	Количество заказываемых контейнеров определенного размера с химикатом	Положительное целое	3	
Емкость	Объем химиката в заказываемом контейнере	Целое	6	
Единица измерения	Единицы измерения, в которых указывается количество заказываемого химиката	Буквенные символы	10	Граммы, килограммы, миллиграммы и т. п.

Пример словаря данных.

АНАЛИЗ ДАННЫХ

Таблица данных должна быть согласована с моделями данных. Можно сравнивать эти дополняющие друг друга представления.

Также, ERP может использоваться в качестве основы для создания БД системы. При этом каждая сущность обычно становится таблицей.

Можно использовать CRUD (Create, Read, Update, Delete) таблицу для анализа данных – это один из основных способов поиска недостающих требований. В примере ниже нет поля D для «Сотрудника, разместившего заказ на химикат», что является недостающим требованием, так как влечет за собой неоднозначную трактовку.

Сущность Вариант использования	Заказ	Химикат	Сотрудник, разместивший заказ на химикат	Каталог поставщика
Разместить заказ	C	R	R	R
Изменить заказ	U, D		R	R
Управлять складом химикатов		C, U, D		
Создать отчет по заказу	R	R	R	
Редактировать сотруд- ников, имеющих право размещать заказы			C, U	

СПЕЦИФИКАЦИЯ ОТЧЕТОВ

Сначала нужно определить, какие отчеты уже используются, какие нужны, их формат, стандарты и т.д. Список вопросов есть в Вигерс, стр. 300. Шаблон спецификации отчета – стр. 303.

Некоторые важные моменты:

- Можно предлагать другие варианты: например, добавление параметров; возможность формирования детализированного и высокоуровневого отчета.
- Важно определить, какие данные использует тот или иной отчет.
- Нужно предусматривать возможность расширение. Например, если в отчете есть таблица с отделами, а их количество увеличится, может испортиться общий вид отчета.
- Некоторые схожие отчеты можно объединить в один путем добавления параметров.
- Нужно различать статические и динамические отчеты.
- Прототипы отчетов могут дать наглядное представление.

ПАНЕЛИ МОНИТОРИНГА

Представление происходящего с помощью различных графиков, чисел и т.д.

Обычно обновляются в реальном времени.

Порядок выявления требований к панели мониторинга (полезно и при выявлении требований к отчетам):

- определить, какая информация нужна пользователям для отслеживания.
- определить источники всех представляемых данных.
- выбрать наиболее подходящий тип отображения данных.
- определить оптимальный макет и относительные размеры элементов.
- отобразить детали каждого элемента:

- если динамически обновляется, то как часто?
- какие параметры должны быть доступны для изменения?
- какое нужно форматирование (например, выделение цветом)?
- где нужны полосы прокрутки?
- возможность увеличить элемент панели?
- возможность кастомизировать элементы панели?
- возможность переключаться между графическим и табличным представлением?

Атрибуты качества ПО

ВВЕДЕНИЕ

Атрибуты качества ПО (также – требования к качеству) – основная часть нефункциональных требований. Делятся на внешние и внутренние (см. далее).

Набор основных атрибутов качества ПО зависит от типа проекта:

- встроенные системы – производительность, надежность, эффективность, устойчивость, безопасность, эргономика.
- интернет-приложения и корпоративные приложения – доступность, целостность, функциональная совместимость, масштабируемость, безопасность, производительность, безопасность и удобство использования.
- настольные и мобильные приложения – производительность, безопасность, удобство и простота использования.

Некоторые атрибуты качества присутствуют во всех проектах (см. таблицу ниже).

Внешнее качество	Краткое описание
Доступность	Насколько сервисы системы доступны, когда и где они нужны
Удобство установки	Насколько просто правильно установить, удалить и повторно установить приложение
Целостность	Насколько хорошо система защищает от неточности и потери данных
Совместимость	Насколько просто система может взаимодействовать и обмениваться данными с другими системами и компонентами
Производительность	Как быстро и предсказуемо система реагирует на ввод информации пользователем и на другие события
Надежность	Как долго система работает до первого сбоя
Устойчивость	Как хорошо система реагирует на неожиданные условия работы
Защита	Насколько хорошо система защищает от повреждений
Безопасность	Как хорошо система защищает от неправомерного доступа к приложению и его данным
Удобство использования	Как просто людям научиться использовать систему
Внутреннее качество	Краткое описание
Эффективность	Насколько эффективно система использует ресурсы компьютера
Возможность модификации	Насколько легко обслуживать, модифицировать, улучшать и реструктурировать систему
Переносимость	Насколько легко заставить систему работать в другой операционной среде
Возможность повторного использования	В какой степени компоненты могут использоваться в других системах
Масштабируемость	Как хорошо система справляется с увеличением числа пользователей, транзакций, серверов и других расширений
Проверяемость и тестируемость	Как быстро разработчики и тестировщики могут подтвердить, что система реализована правильно

ПЕРВИЧНОЕ ИЗУЧЕНИЕ АТТРИБУТОВ

1. Использование широкого списка атрибутов.
2. Сокращение списка.
3. Определение приоритетов атрибутов (см. таблицку ниже).
4. Выявление конкретных ожиданий по каждому требованию (лучше отправить список вопросов заранее; можно спросить, что считается неприемлемым. Вопросы должны быть понятными, например, в какое время дня система наиболее загруженная, сколько времени будет работать пользователь в среднем с системой и т.д.
5. Определение качественно структурированных требований к качеству:
 - а. Следование SMART: S – Specific (конкретные), M – Measurable (измеримые), A – Attainable (достижимые), R- Relevant (актуальные), T – Time-sensitive (ограниченные по времени).

- б. Для каждого требования определить критерий соответствия, выраженный в цифрах.

Атрибут	Оценка	Доступность	Целостность	Производительность	Надежность	Устойчивость	Безопасность	Удобство использования	Проверяемость и тестируемость
Доступность	2		^	^	^	<	^	^	<
Целостность	6			<	<	<	^	<	<
Производительность	4				<	<	^	^	<
Надежность	2					<	^	^	^
Устойчивость	1						^	^	<
Безопасность	7							<	<
Удобство использования	5								<
Проверяемость и тестируемость	1								

Пример таблицы для расстановки приоритетов. «<» - требование в строке важнее, «^» - требование в столбце важнее.

ВНЕШНИЕ АТТРИБУТЫ КАЧЕСТВА

Внешние атрибуты качества описывают характеристики, наблюдаемые при выполнении ПО.

Доступность (availability)

Это запланированное время доступности, в течение которого система действительно доступна для использования и полностью работоспособна.

Пример:

AVL-1. Система должна быть доступна как минимум 95% времени с 6:00 до 00:00 каждый день и на 99% с 15:00 до 18:00 по местному пояскому времени GMT+3.

Вопросы для выявления:

- Для каких частей системы критически важно оставаться доступными?
- Каковы бизнес-последствия недоступности системы для ее пользователей?
- Если требуется регулярное обслуживание, то когда его лучше всего планировать? Как это влияет на доступность системы? Какая минимальная и максимальная продолжительность периодов обслуживания? Как во время периодов обслуживания обрабатывать попытки доступа со стороны пользователей?
- Если обслуживание должно выполняться на работающей системе, то какое влияние оно будет оказывать на доступность и как минимизировать это влияние?
- Каковы нужны уведомления пользователей о том, что система стала недоступной?
- У каких частей системы более строгие требования к доступности по сравнению с остальными частями системы?
- Какие зависимости доступности существуют между группами функциональности (например, невозможность принять платеж по кредитной карте из-за недоступности функции авторизации кредитных карт)?

Удобство установки

Включает в себя начальную установку, восстановление после некорректной установки, обновление, переустановка и откат к предыдущей версии, удаление, установка дополнений и т.д.

Пример:

INS-1. Необученный пользователь должен суметь выполнить начальную установку ПО в среднем за 10 минут.

INS-2. Для установки ПО на сервере требуется административные полномочия.

Вопросы для выявления:

- Какие установочные операции должны выполняться без нарушения пользовательского сеанса?
- Какие установочные операции потребуют перезагрузки приложения, компьютера или устройства?
- Что должно приложение выполнить после удачной — или неудачной — установки?
- Какие операции должны выполняться для подтверждения правильности установки?
- Нужна ли пользователю возможность устанавливать, удалять, переустанавливать или исправлять только избранные части приложения? Если да, то какие?
- Какие другие приложения нужно остановить перед выполнением установки?
- Какие права, разрешения и привилегии нужны установщику?
- Как должна система вести себя в случае незавершенной установки, например по причине отключения питания или прерывания пользователем?

Целостность (integrity)

Предотвращение потери информации и сохранение корректности введенной в систему информации.

Пример:

INT-1. Система должна защищать от неправомерного добавления, удаления или изменения данных.

Вопросы для выявления:

- Обеспечение того, чтобы изменения в данных выполнялись целиком или вообще не выполнялись. Это может означать отмену изменений данных, если в середине процесса возникнут неполадки.
- Обеспечение постоянства изменений внесенных в данные.
- Координация изменений, вносимых во многие хранилища данных, особенно если изменения должны выполняться одновременно (скажем, на многих серверах) и в определенное время (скажем в 12:00 Гринвичского времени 1 января в нескольких местах).
- Обеспечение физической безопасности компьютеров и внешних устройств хранения.
- Выполнение архивирования данных. (С какой частотой? Автоматически и/или по требованию? В каких файлах или базах данных? На какие носители? Со сжатием и проверкой или без?)
- Восстановление данных из архива.
- Архивирование данных: каких данных, когда архивировать, насколько долго и с какими требованиями по удалению.
- Защита данных, хранимых или заархивированных в облаке, от людей, у которых не должно быть доступа к ним.

Совместимость (interoperability)

Насколько система готова к обмену данными другим ПО и к интеграции с внешними аппаратными устройствами.

Пример:

IOP-1. Система должна иметь возможность импортировать все возможные химические формулы из ПО SkillWork версии 5.0 и выше.

Можно определить и как функциональное требование. Главное, чтобы требование было озвучено, и не надо его дублировать в разных разделах SRS.

Вопросы для выявления:

- С какими другими системами должна взаимодействовать эта система? Какими сервисами или данными они должны обмениваться?
- Какие стандартные форматы данных нужны для обмена с другими системами?
- Какие особые аппаратные компоненты должны взаимодействовать с системой?
- Какие сообщения или коды система должна получать и обрабатывать от других систем и устройств?
- Какие стандартные протоколы связи необходимы для обеспечения совместимости?
- Как внешние обязательные требования к совместимости должна удовлетворять система?

Производительность (performance)

Скорость реакции системы на пользовательские действия и запросы. Связана с внутренним атрибутом Эффективность (см. далее).

Пример:

PER-1. Антиблокировочная система тормозов должна информировать о скорости колеса каждые 2 мск, с погрешностью в 0.1 мск (СКО).

Надежность (reliability)

Вероятность работы ПО без сбоев в течение определенного времени. Характеризуется процентом успешно завершенных операций или средним временем наработки на отказ.

Пример:

REL-1. Не более 5 из 1000 экспериментов могут завершиться неуспешно из-за неполадок ПО.

Вопросы для выявления:

- Как вы будете определять, достаточно ли надежна система?
- Каковы должны быть последствия сбоя при выполнении определенных операций в системе?
- Что вы будете считать критическим отказом, а что — всего лишь досадной неприятностью?
- При каких условиях сбой может иметь серьезные последствия для ваших бизнес-операций?
- Никому не нравится сталкиваться со сбоем системы, но есть ли какие-то части системы, которые совершенно точно должны быть чрезвычайно надежными?
- Если система станет неработоспособной, сколько времени вы сможете обойтись без нее, пока это заметно не скажется на ваших бизнес-операциях?

Устойчивость (robustness)

Уровень, до которого система продолжает корректно работать при вводе неверных данных, подключение сбойных внешних устройств и т.д. Также характеризует скорость восстановления после сбоя.

Пример:

ROB-1. Если при работе с текстом произошел сбой, ПО должно восстановить все изменения, внесенные не раньше, чем за 1 минуту до сбоя, и отобразить их при следующем запуске программы.

Сохранность (safety)

Система не должна нанести вред пользователю и имуществу.

Пример:

SAF-1. Система управления химическим реактором данных должна выключить подогрев воды и дать предупредительный сигнал, если температура в корпусе реактора повышается быстрее, чем 5 К в минуту.

Вопросы для выявления:

- При каких условиях использование этого продукта может нанести вред человеку? Как система может обнаружить эти условия? Как нужно реагировать на них?
- Какая максимально разрешенная частота отказов, способная вызывать повреждения?
- Какие виды отказов могут приносить вред людям или имуществу?
- Какие действия оператора могут непреднамеренно нанести вред людям или имуществу?
- Есть ли какие-то режимы работы, которые могут представлять риск для людей или имущества?

Безопасность (security)

Блокировка неавторизованного использования, защита от хакерских атак и т.д.

Пример:

SEC-1. Система должна заблокировать учетную запись пользователь за 5 неверных попыток авторизоваться в течение 5 минут.

SEC-2. Дверь, разблокированная с помощью бейджика безопасности, должна оставаться открытой еще 8.0 секунд с погрешностью ± 0.1 секунды.

Вопросы для выявления:

- Какие конфиденциальные данные нужно защитить от неправомерного доступа?
- Кто уполномочен просматривать конфиденциальные данные? И кто конкретно не уполномочен?
- При каких бизнес-условиях или в каких временных рамках полномочным пользователям предоставляется доступ к функциональности?
- Какие проверки должны выполняться для проверки того, что пользователь выполняет приложение в безопасной среде?
- Как часто должна антивирусная программа выполнять сканирование на предмет обнаружения вредоносного ПО?
- Должен ли применяться какой-то особый метод проверки подлинности пользователей?

Удобство использования (usability)

Характеризует легкость обучения в работе программы, спец. возможности для профессионалов, эффективное и необременительное использование.

Возможные приемы:

Легкость изучения	Простота использования
Подробные подсказки	Быстрые клавиши
Мастера	Развитые настраиваемые меню и панели инструментов
Видимые команды меню	Много вариантов доступа к одной и той же функции
Содержательные сообщения на понятном для пользователя языке	Автозавершение ввода
Справочная система и всплывающие подсказки	Автоисправление ошибок
Похожесть на другие аналогичные системы	Запись макросов и возможности создания сценариев (скриптов)
Отображение ограниченного числа команд и виджетов	Возможность переносить информацию из предыдущей транзакции
	Автоматическое заполнение полей форм
	Интерфейс командной строки

Пункты к обсуждению:

- среднее время, необходимое конкретному типу пользователей, для правильного выполнения определенной задачи;
- сколько транзакций может правильно выполнить пользователь за заданный период времени;
- какую долю набора задач пользователь может правильно выполнить без посторонней помощи;
- сколько ошибок делает пользователь при выполнении задачи;
- сколько попыток требуется пользователю для выполнения определенной задачи, например поиска определенной функции, спрятанной в глубинах меню;
- задержка или время ожидания при выполнении задачи;
- число операций (щелчков мыши, нажатий клавиш, жестов на сенсорном экране), необходимых для получения нужной информации или выполнения задачи.

Пример:

USE-1. Все функции меню File должны иметь определенные быстрые клавиши, нажимаемые одновременно с Ctrl. Командам меню, которые также присутствуют в MS Word, должны соответствовать тем же быстрым клавишам, что и в MS Word.

ВНУТРЕННИЕ АТРИБУТЫ КАЧЕСТВА

Эти свойства не наблюдаются напрямую при выполнении работы. Они важны для разработчиков и тех. поддержки.

Эффективность (efficiency)

Тесно связана с Производительностью (см. выше).

Пример:

EFF-1. Система должна отображать предупреждение оператору, когда нагрузка достигает 80% максимальной плановой мощности.

Вопросы для выявления:

- Какое число одновременно работающих пользователей ожидается сейчас и в будущем?
- Насколько могут ухудшиться время отклика и другие показатели производительности, прежде чем это начнет серьезно отрицательно сказываться на работе пользователей и бизнес-операциях?
- Сколько операций система должна выполнять одновременно в нормальных и экстремальных условиях работы?

Модифицируемость (modifiability)

Показывает, насколько просто разобраться в дизайне и коде ПО, изменять его и расширять.

Пример:

MOD-1. Вложенность вызываемых функций не должна превышать два уровня.

SUP-1. Принтер должен отображать сообщение об ошибке, если новые картриджи не вставлены.

Портируемость (portability)

Характеризует меру усилий для переноса ПО из одной операционной среды в другую.

Примеры:

POR-1. Модификация версии ПО для iOS с тем, чтобы оно могло работать под Android, должно требовать изменения не более 10% кода.

POR-2. Средство миграции платформы должно переносить индивидуализированные профили пользователей без каких-либо усилий со стороны пользователей.

Вопросы для выявления:

- На каких других платформах должно работать ПО сейчас и в будущем?
- Какие части продукта должны разрабатываться с расчетом на более высокую переносимость по сравнению с другими его частями?
- Какие файлы данных, программные компоненты и другие элементы системы должны быть переносимыми?
- Какие другие атрибуты качества могут пострадать при реализации переносимости в системе?

Возможность повторного использования (reusability)

Пример:

REU-1. Функции ввода химических структур должны быть спроектированы так, чтобы их удавалось повторно использовать на уровне объектного кода в других приложениях.

REU-2. Должны быть использованы стандартные шаблоны проектирования GRASP.

Вопросы для выявления:

- Какие существующие требования, модели, компоненты дизайна, данные или тесты можно повторно задействовать в данном приложении?
- Какая функциональность, имеющаяся в связанных приложениях, удовлетворяет некоторым требованиям к данному приложению?
- Какие части этого приложения хорошо подходят для повторного использования в других местах?
- Какие особые действия нужно предпринять, чтобы части данного приложения можно было бы использовать повторно?

Масштабируемость (scalability)

Способность системы расти – обслуживать больше пользователей, серверов, данных и т.д. без потери производительности и корректности. Конечно, здесь важно железо, но есть и программная часть: оптимизация алгоритмов, распараллеливание процессов и т.д.

Примеры:

SCA-1. Надо обеспечить возможность увеличения пропускной способности аварийной телефонной системы с 500 до 2500 звонков в день в течение 12 часов.

Вопросы для выявления:

- Какова оценка числа общего числа и числа одновременных пользователей, которое должна обслуживать система через несколько месяцев, кварталов или лет?
- Не могли бы вы описать, как и почему в будущем могут вырасти потребности в мощностях хранения данных?
- Какие минимально приемлемые критерии производительности должны быть удовлетворены независимо от числа пользователей?
- Каковы планы будущего роста в плане числа серверов, центров обработки данных или числа установленных экземпляров системы?

Тестируемость (verifiability)

Характеризует легкость проверки корректной работы системы.

Пример:

VER-1. У тестирующей должна быть возможность конфигурировать, какие результаты будут записываться во время тестирования.

VER-2. Максимальная цикломатическая сложность (количество логических ответвлений в модуле исходного кода) модуля не должна превышать 20.

- Как можно подтвердить, что определенные вычисления дают ожидаемые результаты?
- Есть ли какие-то части системы, не обеспечивающие детерминированных результатов, из-за чего сложно определить, правильно ли они работают?
- Возможно ли создать набор тестовых данных, которые с высокой вероятностью позволят выявить какие-либо ошибки в требованиях или их реализации?
- Какие стандартные отчеты или другие результаты можно использовать для проверки того, что система возвращает правильные результаты?

ЯЗЫК PLANGUAGE

Определение ключевых параметров (тегов) для каждого требования. В результате этого требования становятся более понятным, но и более объемным (см. Вигерс, стр. 340).

СВЯЗЬ АТТРИБУТОВ

На практике нельзя удовлетворить всем требованиям сразу. Приходится искать компромиссы. В таблице ниже описана взаимосвязь атрибутов. + (-) значит, что улучшение атрибута в строке положительно (отрицательно) сказывается на атрибуте в столбце.

	Доступность	Эффективность	Удобство установки	Целостность	Совместимость	Возможность модификации	Производительность	Переносимость	Надежность	Возможность повторного использования	Устойчивость	Защита	Масштабируемость	Безопасность	Удобство использования	Проверяемость и тестируемость
Доступность								+		+						
Эффективность	+			-	-	+	-			-		+		-		
Удобство установки	+							+					+			
Целостность		-		-		-			-		+		+	-	-	
Совместимость	+	-	-			-	+	+		+	-		-			
Возможность модификации	+	-				-		+	+			+				+
Производительность		+		-	-		-			-		-		-		
Переносимость		-		+	-	-			+				-	-	+	
Надежность	+	-	+	+	+	-				+	+		+	+	+	
Возможность повторного использования		-	-	+	+	-	+						-		+	
Устойчивость	+	-	+	+	+	-		+			+	+	+	+		
Защита		-	+	+		-				+			+	-	-	
Масштабируемость	+	+		+		+	+	+		+						
Безопасность	+		+	+		-	-	+		+	+			-	-	
Удобство использования		-	+			-	-	+		+	+				-	
Проверяемость и тестируемость	+		+	+				+	+	+	+		+	+		

ПЕРЕХОД ОТ АТТРИБУТОВ КАЧЕСТВА К ТЕХНИЧЕСКОЙ ИНФОРМАЦИИ

Атрибуты качества могут переходить в функциональные требования и определять направление дизайна. Например, безотказная работа мед. оборудования приводит к наличию резервного питания

(архитектура) и к возможности оповещения при переходе на резервное питание (функциональное требование).

ОГРАНИЧЕНИЯ

Некоторые считают атрибуты качества ограничениям (в принципе, так и есть – атрибуты качества часто являются источниками ограничениям). Однако их можно выделить в отдельный класс нефункциональных требований. Ограничения часто связаны со стандартами разработки, совместимость с другими продуктами (форматы данных). Также источник ограничений – технические решения и решения руководства.

Ограничения – это предполагаемое решение, и поэтому его надо тщательно исследовать с помощью дополнительных вопросов «почему?», чтобы добраться до исходного требования.

Примеры:

CON-1. Для реализации продукта может использоваться ПО с лицензией GNU.

CON-2. Приложение должно использовать Microsoft .Net Framework 4.5.2.

CON-3. Все используемые в приложении текстовые данные должны храниться в виде XML-файлов.

Использование прототипов

НАЗНАЧЕНИЕ

- Прояснение, формулировка и утверждение требований.
- Исследование альтернативных решений.
- Создание подмножества, которое в конечном итоге превратиться в продукт.

КЛАССИФИКАЦИЯ

По назначению:

- Модель (mockup) – предполагаемый интерфейс пользователя (горизонтальный прототип).
- Экспериментальный образец (proof of concept) – срез функциональности приложения (вертикальный прототип).

По использованию:

- Одноразовые – для выяснения деталей (низкое качества кода, потом не используется ни в коем случае).
- Эволюционный – в дальнейшем превращается в продукт (в agile примерно так и делается).

По типу:

- Бумажные (на бумаге, планшете и т.д.).
- Электронные.

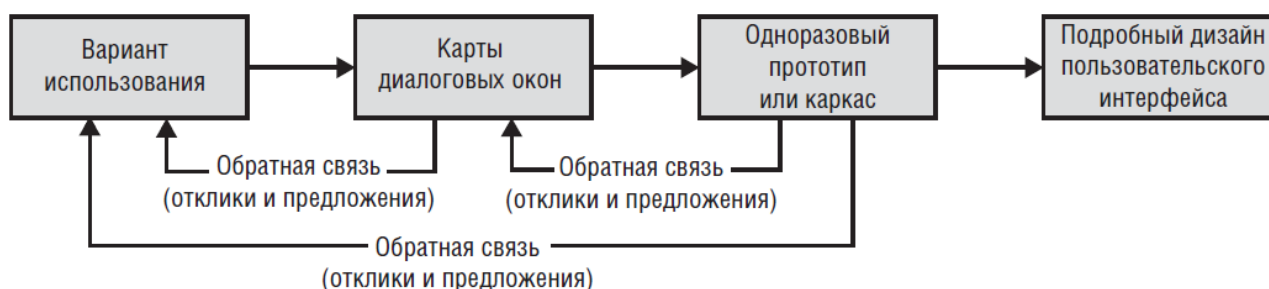
	Одноразовый	Эволюционный
Модель	Прояснение и уточнение пользовательских и функциональных требований Выявление пропущенной функциональности Исследование возможных вариантов пользовательского интерфейса	Реализация базовых пользовательских требований Реализация дополнительных пользовательских требований по приоритетам Реализация и доработка веб-сайтов Адаптация системы к быстро меняющимся требованиям бизнеса
Экспериментальный образец	Демонстрация технической осуществимости Оценка производительности Получение информации для уточнения оценок	Реализация и наращивание ключевой многоуровневой функциональности и уровней коммуникации Реализация и оптимизация основных алгоритмов Тестирование и настройка производительности

Вопросы для прототипов:

- Реализует ли прототип все необходимые функции так, как вы этого ожидали?
- Не упущена ли в прототипе какая-либо функциональность?
- Заметили ли вы какие-либо условия ошибки, не учтенные в прототипе?
- Нет ли в прототипе каких-либо ненужных функций?
- Насколько логичной и полной вам кажется навигация?
- Не показалось ли вам, что можно упростить каких-либо задачи, в которых слишком много интерактивных шагов?
- Оказывались ли вы в ситуации, когда не знали, что делать дальше?

СОЗДАНИЕ ИНТЕРФЕЙСА ЧЕРЕЗ ПРОТОТИПЫ

Варианты использования и карты диалоговых окон описаны выше.



РИСКИ ПРИ СОЗДАНИИ ПРОТОТИПА

- Подталкивание к выпуску продукта.
- Отвлечение на детали.
- Нереалистичные ожидания пользователей.
- Слишком много усилий на создание.

Главное, при создании прототипов:

- заранее включать создание прототипов в график работы.
- четкая формулировка цели каждого прототипа.
- минимум усилий в одноразовые прототипы.

Расстановка приоритетов требований

ВВЕДЕНИЕ

В реальных проектах практически никогда не реализуется весь функционал, который предполагалось реализовать изначально.

Если разработка происходит итеративно, то важно знать, какие требования выполнить в ближайшем релизе, а какие можно отложить на более далекие версии.

Для этого нужно расставлять приоритеты требований.

Важно отслеживать связанные требования. Также, нужно оставлять низкоприоритетные требования, реализация которых существенно влияет на архитектуру и может предотвратить от переделок в будущем.

МЕТОДЫ

1. Парное сравнение и ранжирование - как турнирная таблица. При большом количестве требований становится затруднительным. Поэтому требования лучше объединять в связанные группы.
2. Таблица Важно/Не важно + Срочно/Не срочно. В первую очередь делать срочно и важно. Срочно/Не важно, как правило, вовсе не нужно. И не надо забывать о Важно/Не срочно.
3. MoSCoW:
 - a. M - Must (ключевое, должно быть включено),
 - b. S - Should (важно, и желательно, должно быть включено, но не определяет успех),
 - c. C - Could (при возможности - включить),
 - d. W - Won't (не обязательно).

Недостаток - часто все попадает в Must, и заинтересованные лица думают, отчасти, справедливо, что S, C, W не будут реализованы вовсе.
4. 100 dollar - заинтересованные лица как бы платят за требования свои "деньги", и в конце реализуются самые востребованные требования. Здесь лучше добавить анонимность, иначе можно начать хитрить: если человек видит, что его требование уже «проплатили», он может не платить за него, отдавая «деньги» менее приоритетным требованиям.
5. Таблица ценности, стоимость и рисков. Делаются таблицы для групп требований, где определяются эти три параметра.
 - a. Клиенты определяют Выгоду от 1 до 9 каждой группы связанных требований.

- b. Клиенты определяют Ущерб, который понесет бизнес или клиент, если функция не будет выполнена (от 1 до 9).
- Проиграет ли ваш продукт по сравнению с другими, в которых эта функция есть?
 - Будут ли какие-либо юридические или контрактные последствия?
 - Не нарушите ли вы какую-либо норму права или отраслевой стандарт?
 - Не лишит ли это пользователей возможности выполнять какие-либо необходимые или ожидаемые действия?
 - Намного ли сложнее добавить эту функцию позже, в качестве усовершенствования?
 - Возникнут ли проблемы из-за того, что отдел маркетинга уже обещал эту функцию, некоторым клиентам?
- c. Рассчитывается Ценность как $2 \cdot \text{Выгода} + 1 \cdot \text{Ущерб}$.
- d. Стоимость реализации определяют разработчики (от 1 до 9).
- e. Также разработчики определяют Риск (вероятность не сделать с первого раза), тоже от 1 до 9.
- f. Потом рассчитывается приоритет как $\text{Ценность} / (\text{Стоимость} + \text{Риски})$.

Относительный вес		2	1		1		0,5			
Функция		Относительная выгода	Относительный ущерб	Общая ценность	Ценность, %	Относительная стоимость	Стоимость, %	Относительный риск	Риск, %	Приоритет
1.	Распечатка списка данных по безопасности материалов	2	4	8	5,2	1	2,7	1	3,0	1,22
2.	Запрос о статусе заказа поставщика	5	3	13	8,4	2	5,4	1	3,0	1,21
3.	Создание отчета о инвентаризации склада химических	9	7	25	16,1	5	13,5	3	9,1	0,89
4.	Просмотр истории конкретного контейнера с химикатом	5	5	15	9,7	3	8,1	2	6,1	0,87
5.	Поиск химиката по каталогам поставщиков	9	8	26	16,8	3	8,1	8	24,2	0,83
6.	Поддержка списка опасных химических	3	9	15	9,7	3	8,1	4	12,1	0,68

Утверждение требований

ВВЕДЕНИЕ

Проверка требований может значительно улучшить SRS, что уменьшит издержки в будущем.

Основной вид утверждения требований – это рецензирование. Рецензирование может быть неформальным (попросить коллег почитать), так и официальным (экспертиза).

Неформальное рецензирование – просто попросить своих друзей и коллег просмотреть SRS (конечно, по каждой части нужно обращаться к соотв. специалисту).

Утверждение нужно, чтобы:

- в спецификации требований к ПО должным образом описаны предполагаемые возможности и характеристики системы, которые удовлетворяют потребности различных заинтересованных в проекте лиц;
- требования к ПО точно отражают бизнес-требования, системные требования, бизнес-правила и другие источники;
- требования полные, реализуемые и проверяемые;
- все требования необходимы, а всей их совокупности достаточно для достижения бизнес-целей;
- все требования согласованы друг с другом;
- требования обеспечивают качественную основу для дизайна и создания ПО.

ЭКСПЕРТИЗА

Участники:

1. Автор спецификации (SRS).
2. Люди-источники информации, которая легла в основу требований.
3. Разработчики и тестировщики (возможность реализовать и проверить требование).
4. Люди, отвечающие за работу продуктов, взаимодействующих с разрабатываемым продуктом (требования к внешним интерфейсам).

Роли:

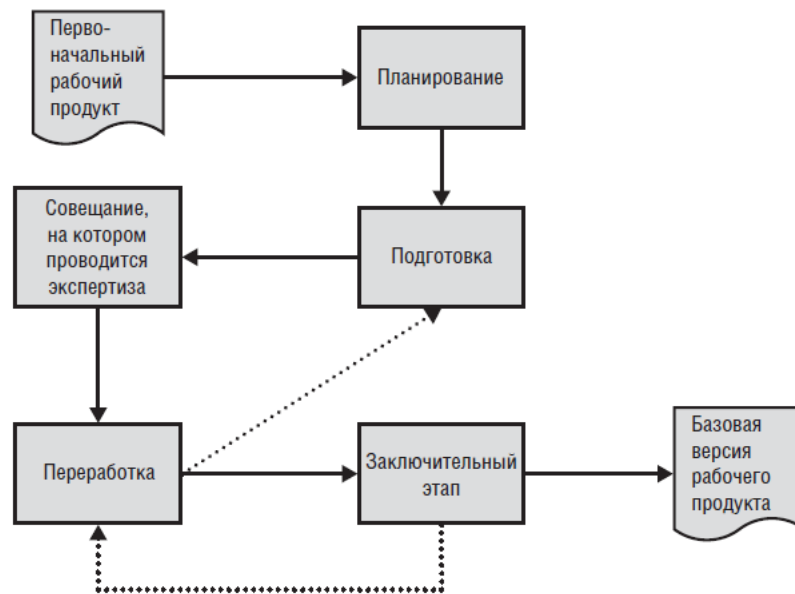
1. Автор. Слушает, отвечает на вопросы, но не вступает в дебаты и сам не проверяет. Организует совещание и процедуру рецензирования. Как правило, это и есть автор спецификации.
2. Координатор. Подготавливает все за несколько дней до совещания: материалы SRS, распечатка контрольных листов и т.д. Ведет совещание и отвечает за его организацию.
3. Читатель. Зачитывает все требования и свою интерпретацию. Другие соглашаются или дискутируют. В менее формальных совещаниях это может делать секретарь, просто пробегаая по пунктам SRS.
4. Секретарь. Ведение протокола.

Документ должен быть тщательно проверен до начала совещания (орфография, стиль, отсутствие пробелов).

ЭТАПЫ ЭКСПЕРТИЗЫ

1. Планирование. Автор и координатор определяют состав участников. Обозначение общего графика работы так, чтобы никого это не напрягало (так как и так все заняты).
2. Подготовка. Индивидуальное чтение и проверка по контрольному листу (примерно половина времени совещания).
3. Совещание. Читатель зачитывает требования, дискуссия, правка, обозначение проблем.

4. Переработка. Автор дорабатывает замечания совместно с ответственными людьми за эти требования (см. НИЖЕЕ).
5. Заключительный этап. Координатор проверяет, что все выявленные дефекты SRS были устранены. Если нет, то возвращение на этап 2.



КОНТРОЛЬНЫЙ ЛИСТ ДЕФЕКТОВ

Полнота

- ☐ Включены ли в требования все известные потребности клиента или системы?
- ☐ Отсутствует ли в требовании какая-либо необходимая информация? Если да, помечена ли она значком «TBD»?
- ☐ Определены ли все алгоритмы, соответствующие функциональным требованиям?
- ☐ Определены ли все внешнее оборудование, ПО и интерфейсы взаимодействия?
- ☐ Задokumentировано ли ожидаемое поведение системы для всех предполагаемых ошибочных условий?
- ☐ Обеспечивают ли требования адекватную основу для дизайна и тестирования?
- ☐ Указан ли приоритет реализации каждого требования?
- ☐ Находится ли каждое требование в рамках проекта, выпуска или итерации?

Корректность

- ☐ Конфликтуют ли какие-либо требования с другими требованиями или повторяют их?
- ☐ Написано ли каждое требование ясным, точным и недвусмысленным языком?
- ☐ Можно ли проверить каждое требование с помощью тестирования, демонстрации, просмотра или анализа?
- ☐ Все ли перечисленные сообщения об ошибках уникальны и выразительны?
- ☐ Являются ли все требования действительно требованиями, а не решениями или ограничениями?
- ☐ Можно ли реализовать все требования, не выходя за рамки установленных ограничений?

Атрибуты качества

- ☐ Все ли цели по производительности, безопасности и защите сформулированы надлежащим образом?
- ☐ Все ли остальные атрибуты качества задokumentированы и оценены количественно с указанием приемлемых компромиссов?
- ☐ Все ли критичные по времени функции определены и задан график реализации для них?
- ☐ Решены ли надлежащим образом вопросы интернационализации и локализации?
- ☐ Все ли требования к качеству поддаются измерению?

Организация и возможность отслеживания

- ☐ Все ли требования организованы логичным и доступным образом?
- ☐ Корректны ли все перекрестные ссылки на другие требования и документы?
- ☐ Написаны ли все ли требования на одном и том же надлежащем уровне детализации?
- ☐ Каждое ли требование идентифицировано уникально и корректно?
- ☐ Указана ли для каждого функционального требования ссылка на источник (например, на системное требование или бизнес-правило)?

Другие проблемы

- ☐ Не пропущены ли какие-то варианты использования или потоки процессов?
- ☐ Не пропущены ли в вариантах использования какие-либо альтернативные направления, исключения или другая информация?
- ☐ Определены ли все бизнес-правила?
- ☐ Не упущены ли какие-либо наглядные модели, которые необходимы для ясности и полноты?
- ☐ Не пропущены ли какие-либо спецификации отчетов и полны ли они?

- Создается для каждого типа продукта отдельно.
- Желательно использовать шаблон.
- Можно делать разные списки для разных участников.

СОВЕТЫ ПО РЕЦЕНЗИРОВАНИЮ

1. Каждый участок документа должны проверять "нужные" люди.
2. Рецензировать можно на этапе создания требований, по мере создания SRS.
3. Заранее планировать временной график, так как все, по сути, являются занятыми людьми.
4. Если требуется много этапов, нужно всегда отмечать внесенные изменения.
5. Определять приоритеты разных частей документа.
6. Важно убедиться, что все проверяющие понимают, что они должны выявлять дефекты, а не отстаивать свою точку зрения.

ТЕСТИРОВАНИЕ ТРЕБОВАНИЙ

Тесты помогают выявить недостатки в требованиях.

Концептуальные модели для тестирования требований – это тесты, которые не зависят от реализации (абстрактные). Можно сразу формировать на этапе создания пользовательских требований.

Пример:

- пользователь вводит номер заказа, который хочет просмотреть, этот заказ существует, пользователь разместил заказ. Ожидаемый результат: отображение подробностей заказа;
- пользователь вводит номер заказа, который хочет просмотреть, этот заказ не существует. Ожидаемый результат: появляется сообщение: «К сожалению, заказ не найден»;
- пользователь вводит номер заказа, который он хочет просмотреть, этот заказ существует, но он размещен не этим пользователем. Ожидаемый результат: появляется сообщение: «К сожалению, это не ваш заказ».

Также, например, отслеживание путей перемещения по карте диалоговых окон можно найти пропущенные или некорректные требования. Это тоже часть концептуальных тестов.

Бизнес-требования проверяются через связанные варианты использования и функциональные требования.

КРИТЕРИИ ПРИЕМКИ

Критерии приемки определяют, соответствует ли разработанное ПО требованиям SRS. Это не функциональные и не модульные тесты. Критерии приёмки ориентированы на выполнение бизнес-целей. В общем, это может быть:

1. Наличие приоритетной функциональности (естественно, работающей правильно).
2. Обязательные атрибуты качества (метрики), которые должны удовлетворяться.
3. Остающиеся дефекты и недоработки (например, ПО принимают, если количество дефектов не превышает 5%)
4. Выполнение юридических и нормативных требований (например, в отчетах).
5. Проектные требования, не относящиеся к продукту (наличие обучающих материалов и т.д.).

Приемочные тесты составляют основную часть критериев приемки. Направлены на нормальные варианты использования и их исключения.

В гибкой методологии разработке они пишутся вместо требований, и должны описывать все варианты использования.

Повторное использование требований

ТРИ ИЗМЕРЕНИЯ

1. Объем повторного использования (при этом использование существующего функционала можно делать через API с составлением соответствующей документацией). В принципе, может использоваться вплоть до кусков кода, реализующих требование.
2. Объем изменений (от простого копипаста до серьезных изменений)
3. Механизм повторного использования: копипаст требования, копирование из библиотеки требований, ссылка на источник. Лучше использовать специальные средства отслеживания изменений.

ТИПЫ ИНФОРМАЦИИ ДЛЯ ПОВТОРНОГО ИСПОЛЬЗОВАНИЯ

Область повторного применения	Кандидаты активов требований на повторное использование
В продукте или приложении	Пользовательские требования, отдельные функциональные требования в вариантах использования, требования к производительности, требования к удобству использования, бизнес-правила
В рамках семейства продуктов	Бизнес-цели, бизнес-правила, модели бизнес-процессов, контекстные диаграммы, карты экосистемы, пользовательские требования, базовые функции продукта, профили заинтересованных лиц, описания классов пользователей, архетипы пользователей, требования к удобству использования, требования к безопасности, требования к соответствию нормативам и законам, сертификационные требования, модели и определения данных, приемочные тесты, словарь терминов
В рамках предприятия	Бизнес-правила, профили заинтересованных лиц, описания классов пользователей, архетипы пользователей, требования к безопасности, словарь терминов
В рамках предметной области	Модели бизнес-процессов, функции продукта, пользовательские требования, описания классов пользователей, архетипы пользователей, приемочные тесты, словарь терминов, модели и определения данных, бизнес-правила, требования к безопасности, требования к выполнению нормативов и законов
В операционной среде или платформе	Ограничения, интерфейсы, инфраструктуры функциональности, необходимой для поддержки определенных типов требований (например, к генератору отчетов)

Популярные сценарии повторного использования:

1. Семейства продуктов (схожая функциональность, интерфейс, hotkeys и т.п. упрощает жизнь пользователям и разработчикам).
2. Реинжиниринг и замена системы (понятно, использовать уже реализованные части).
3. Другие примеры (см. таблицу ниже).

Возможность повторного использования	Примеры
Бизнес-процессы	Часто бизнес-процессы бывают одинаковыми во многих организациях и должны одинаково обслуживаться программным обеспечением. Многие организации поддерживают набор описаний бизнес-процессов, которые можно использовать в разных ИТ-проектах
Распределенные среды	Часто одна система развертывается многократно с небольшими вариациями. Такая ситуация очень часто наблюдается в розничных магазинах и на складах. При развертывании всех экземпляров повторно используется общий набор требований
Интерфейсы и интеграция	Часто возникает потребность в повторном использовании требований к интерфейсам и процессам интеграции. Например, в больницах у большинства вспомогательных систем должны быть интерфейсы входного и выходного взаимодействия с системами приема, выписки и перевода. Это также верно в отношении финансовых интерфейсов системы управления ресурсами предприятия
Безопасность	Аутентификация пользователей и требования к безопасности часто одинаковы во многих системах. Например, у систем может быть общее требование, что все продукты должны поддерживать единый вход с использованием Active Directory для аутентификации пользователей
Общие функции приложений	Бизнес-приложения часто содержат общую функциональность, для которой можно повторно использовать требования, и даже всю реализацию целиком. Вот несколько примеров: операции поиска, печать, файловые операции, пользовательские профили, отмена и повтор операции и форматирование текста
Аналогичные продукты для разных платформ	Единый базовый набор требований используется даже при наличии некоторых подробных требований и/или отличий в дизайне пользовательского интерфейса, обусловленных платформой. В качестве примера можно привести приложения, которые работают в среде Mac и Windows или в iOS и Android
Стандарты, нормативы и правовые нормы	Во многих организациях разработаны наборы стандартов — часто основывающихся на нормативно-правовых актах, — которые определены как набор требований. Их можно повторно использовать между проектами. Примеры: Стандарты ADA для дизайна с расчетом на ограниченные возможности и требования закона об ответственности и переносе данных о страховании здоровья граждан (HIPAA),

СХЕМЫ ТРЕБОВАНИЙ

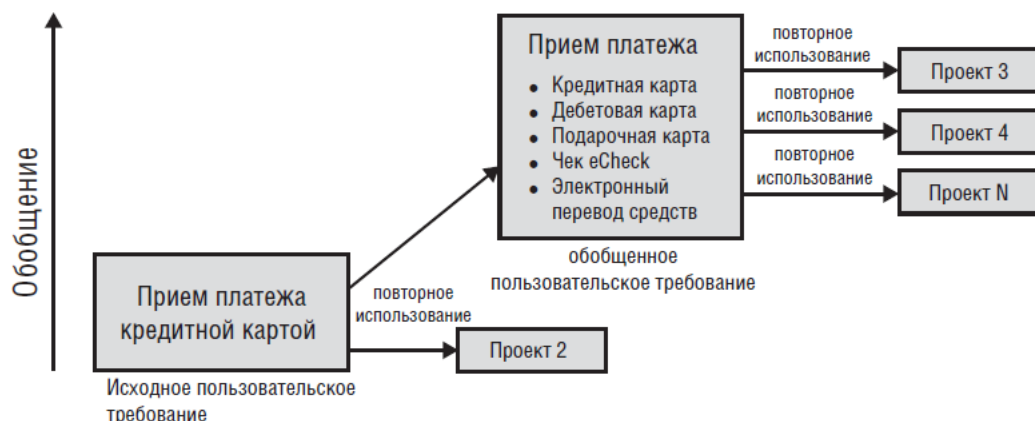
Существуют схемы требований - схемы, по которым просто создавать определенный тип требований. Можно создать БД таких схем внутри организации.

В некоторых системах требования существуют уже готовые шаблоны.

Факторы, способствующие успешному повторному использованию:

1. Наличие хранилища требований.

2. Качественные требования (желательно – обобщенные, см. схему ниже).
3. Словари терминов и данных.
4. Организационная культура повторного использования.



От требований к разработке

1. Примерно 15% времени проекта уходит на разработку требований. Нужно учиться оценивать правильно, всегда сравнивая цифры с реальностью. Нужно знать производительность членов команды и другие важные факторы при планировании.
2. По требованиям можно оценивать объем работ (количество отдельно тестируемых требований в waterfall и баллы историй в Agile).
3. Дизайн ПО:
 - разработаете надежную архитектуру подсистем и компонентов, которая обеспечит поддержку будущих изменений;
 - определите ключевые классы объектов или функциональные модули, которые необходимо создать, определив их интерфейсы, функции и взаимодействие с другими элементами;
 - убедитесь, что в дизайн включены все функциональные требования и что он не содержит ненужной функциональности;
 - определите предполагаемую функциональность каждой единицы кода, которая следует основному принципу дизайна: сильная связанность, слабая связь и сокрытие информации (McConnell, 2004);
 - удостоверьтесь, что дизайн учитывает все условия исключения, которые могут возникнуть;
 - убедитесь, что дизайн будет отвечать заданным критериям производительности, безопасности и другим целям по качеству;
 - убедитесь, что существующие компоненты можно использовать повторно;
 - определите и соблюдайте все ограничения, оказывающие значительное влияние на дизайн программных компонентов.
4. Дизайн интерфейса - модель display-action-response, если придется описывать и дизайн, что маловероятно.

Поведение элементов пользовательского интерфейса

Предварительное условие	Действие пользователя	Реакция
Всегда	Пользователь щелкает ссылку: Home, About the Book, About the Author, Buy the Book, Contact, Pearl Submission Guidelines, Pearl или Submission Terms	Открывается соответствующая страница
Всегда	Пользователь щелкает любую из ссылок Blog	Блог Pearls from Sand открывается в новой вкладке браузера
Всегда	Пользователь вводит или вставляет текст в текстовое поле	Текст пользователя отображается в поле Your Story, также отображается число оставшихся символов
Всегда	Пользователь щелкает флажок I agree	Состояние флажка изменяется по циклу «включен-отключен»
Одно или несколько неверных значений в элементах управления	Пользователь щелкает ссылку Submit	Отображается сообщение об ошибке, о неверном тексте, длине или незаполненных обязательных полях

5. Тестирование должно быть на основе требований, а не на основе кода и дизайна. Тестировать нужно: нормальный сценарии, исключения и обработку ошибок, соответствие атрибутам качества.

Виды тестов:

1. Тест (работа ПО).
2. Экспертиза код (проверка кода на соотв. требованиям).
3. Анализ (обоснование того, как система должна работать в определенных условиях).

УПРАВЛЕНИЕ ТРЕБОВАНИЯМИ

Основные документы процесса управления

Документы процесса для разработки требований	Документы процесса управления требованиями
• Процесс разработки требований • Процедура назначения требований • Процедура назначения приоритетов требованиям • Документ о концепции и границах • Шаблон вариантов использования • Шаблон спецификации требований к ПО • Контрольный список рецензирования требований	• Процесс управления требованиями • Процедура отслеживания состояния требований • Процесс управления изменениями • Шаблон устава совета по управлению изменениями • Контрольный список анализа влияния, оказываемого изменениями • Процедура отслеживания требований

Структура процесса управления



Общая структура процесса управления.

Управление версиями

Нужно всегда четко записывать идентификатор документа. Для отслеживания можно использовать системы контроля версий типа Git. Чтобы знать, кто и когда сделал какие изменения, нужно отмечать хронику изменений документа (в самом начале документа). Все версии требований нужно хранить.

Изначальная базовая версия требований (которая становится такой после рецензирования и утверждения) для каждой версии продукта считается исходной.

У каждого высокоприоритетного (а лучше, у любого) требования, желательно, должны быть атрибуты:

1. Дата создания, автор.
2. Приоритет.
3. Состояние.
4. Происхождение или источник.
5. Логическое обоснование.
6. Номер выпуска или итерации, на которую назначено требование.
7. Лицо, ответственное за внесение изменений в требование.
8. Метод проверки или критерий приемки.

Управление изменениями

МЕТОДЫ УПРАВЛЕНИЯ ОБЪЕМОМ ТРЕБОВАНИЙ

1. Фиксация границ в документе о концепции и границах.
2. Составление прототипов. Правильные прототипы помогают определить границы проекта или конкретной итерации.
3. Правильно откладывать на потом, т.е. говорить "нет".

Не надо замораживать требования для новой системы. Важно дойти до базовой версии, изменения которой будут минимальны.

ПРАВИЛА ВНЕСЕНИЯ ИЗМЕНЕНИЙ

1. Все изменения должны быть согласованы и одобрены уполномоченными лицами (советом, см. ниже).
2. База данных предложенных и принятых изменений требований должны быть доступна всем участникам проекта.
3. Все заинтересованные лица должны уведомляться об одобренных изменениях.
4. Каждое предложение должно быть тщательно проанализировано.

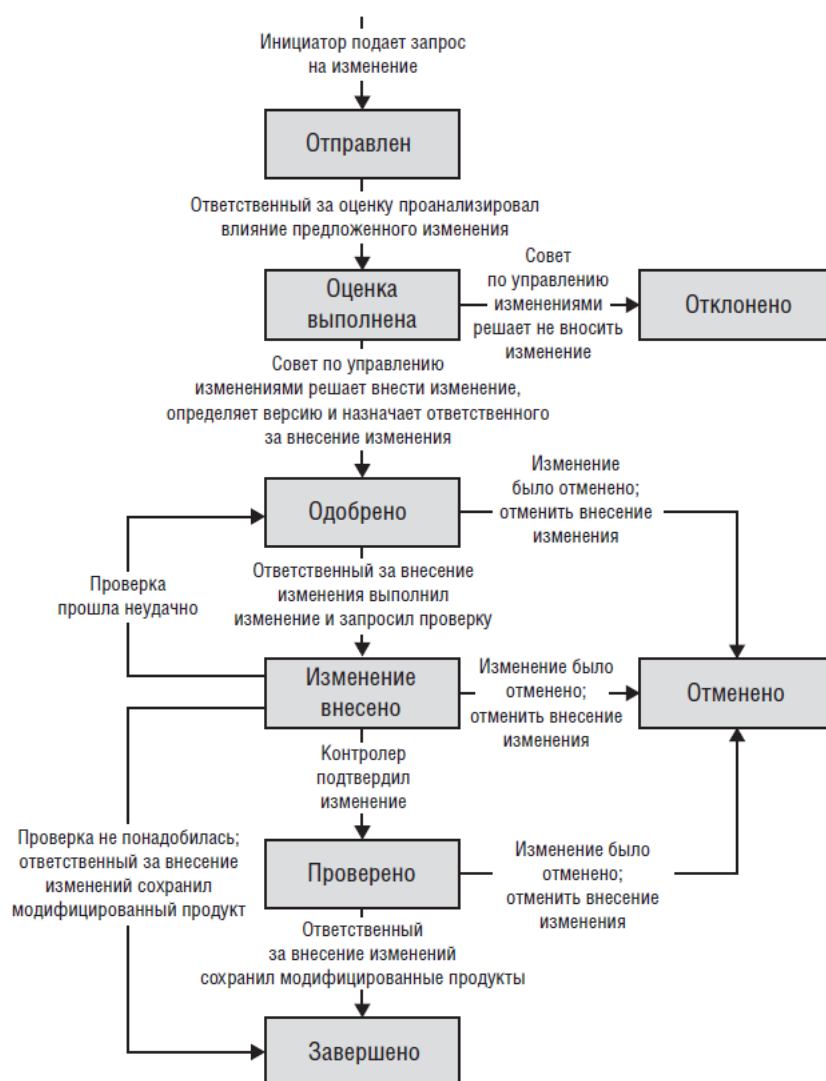
ШАБЛОН ОПИСАНИЯ ПРОЦЕССА УПРАВЛЕНИЯ ТРЕБОВАНИЙ

1. Роли и ответственности лиц.
2. Состояние запроса на изменение.
3. Анализ:
 - a. Анализ запроса (оценка на соотв. бизнес-целям, риска, анализ влияния).
 - b. Принятие советом решения об изменении (в какой версии и итерации, какой приоритет, может быть отклонено и т.д.). Как принято решение, необходимо уведомить все заинтересованные лица.
 - c. Реализация требования в продукте. Тут важно выявить и реализовать все связанные изменения.
 - d. Тестирование и рецензирование.
4. Выход: статус изменения, уведомление всех заинтересованных лиц, фиксация требования в продукте и документации.

У каждого предложенного изменения есть набор атрибутов: дата предложения, инициатор, описание, идентификатор и т.д.

Роль	Описание и обязанности
Председатель совета по управлению изменениями	Возглавляет совет по управлению изменениями; как правило, обладает правом принятия окончательного решения, если совет не приходит к согласию; выбирает ответственных за оценку и внесение изменений
Совет по управлению изменениями	Группа, принимающая решение утвердить или отклонить каждое предложенное изменение для определенного проекта
Ответственный за оценку	Лицо, которое председатель совета просит проанализировать влияние предложенного изменения
Ответственный за внесение изменений	Лицо, отвечающее за внесение изменений в продукт, когда запрос на изменение утвержден
Инициатор	Лицо, подающее новый запрос на изменение
Получатель запросов	Лицо, которому передаются новые запросы на изменение
Контролер	Лицо, определяющее, корректно ли реализовано изменение

Роли и ответственности лиц.



Состояние запроса на изменение.

Можно корректировать, здесь приведен довольно общий и полный шаблон.

СОВЕТ ПО ВНЕСЕНИЮ ИЗМЕНЕНИЙ

1. Состав (представители различных классов заинтересованных лиц: менеджеры продукта, аналитики, разработчики, тестировщики, маркетологи, тех. поддержка и т.д.).
2. Должен быть установлен порядок принятия решений (председатель или голосованием).
3. Способ оповещения нужных специалистов о внесенных изменениях.
4. Пересмотр обязательств.

Есть специализированные средства для управления изменениями.

Прим. SRS для разных версий лучше хранить в системе управления требованиями: там можно смотреть требования как ко всему продукту, так и к конкретной его версии.

ИЗМЕРЕНИЕ АКТИВНОСТИ ВНЕШЕНИЯ ИЗМЕНЕНИЙ

Необходимо измерять активность изменений, чтобы понимать сроки сдачи проекта и качество базовой версии требований.

- общее количество полученных запросов на изменение, открытых и закрытых в настоящее время;
- совокупное число пришедших запросов, включая добавленные, удаленные и измененные требования;
- количество полученных запросов на изменение, исходящих из каждого источника;
- количество изменений, предложенных и внесенных в каждое требование после создания базовой версии;
- общие усилия на обработку и реализацию запросов на изменение.

АНАЛИЗ ИЗМЕНЕНИЙ

Делает один из специалистов, по поручению председателя совета. Результат формируется в виде небольшого отчета, который, желательно, также утвердить в рамках организации.

1. Понимание последствий и рисков (про риски см. ниже).
2. Определение рабочих продуктов, на которые может повлиять изменение.
3. Оценка работ по внедрению изменения.

Вопросы, позволяющие понять возможные последствия предлагаемого изменения:

- ☐ Улучшит или ухудшит ли изменение способность удовлетворять каким-либо бизнес-требованиям?
- ☐ Конфликтуют ли какие-то из требований в базовой версии с предложенным изменением?
- ☐ Конфликтуют ли какие-то из отложенных требований в базовой версии с предложенным изменением?
- ☐ Каковы технические или бизнес-последствия невнесения изменения?
- ☐ Какие могут быть побочные негативные эффекты или другие риски, если изменение не будет внесено?
- ☐ Повлияет ли негативно предложенное изменение на требования к производительности и другие атрибуты качества?
- ☐ Выполнимо ли предложенное изменение в рамках известных технических ограничений и квалификации специалистов?
- ☐ Не потребуются ли для реализации предложенного изменения неприемлемое количество каких-либо ресурсов, необходимых для разработки, тестирования или рабочих сред?
- ☐ Нужно ли приобрести какие-либо средства для реализации и тестирования этого изменения?
- ☐ Как предложенное изменение повлияет на последовательность, зависимости, усилия или продолжительность задач, включенных в настоящее время в план проекта?
- ☐ Потребуется ли создание прототипов или другая информация пользователей для проверки предложенного изменения?
- ☐ Сколько затрат, уже вложенных в проект, будут потеряны, если принять это изменение?
- ☐ Не увеличится ли затрата на единицу продукта при принятии изменения, например за счет увеличения расходов на покупку лицензий на продукты сторонних разработчиков?
- ☐ Повлияет ли изменение на планы по маркетингу, производству, обучению или поддержке клиентов?

Список определения продуктов, на которые могут повлиять предлагаемое изменение:

- ☐ Определите все необходимые изменения пользовательского интерфейса, дополнения или удаления.
- ☐ Определите все изменения, добавления или удаления, которые необходимо внести в отчеты, базы данных или файлы.
- ☐ Определите компоненты дизайна, которые придется создать, изменить или удалить.
- ☐ Определите файлы исходного кода, которые необходимо создать, изменить или удалить.
- ☐ Определите все изменения, которые придется внести в файлы или процедуры построения.
- ☐ Определите существующие тесты модулей, интеграции и системные тесты, которые потребуется изменить или удалить.
- ☐ Оцените необходимое количество новых тестов модулей, интеграции и системных тестов.
- ☐ Определите все экраны справочной системы, обучающие материалы или другую пользовательскую документацию, которую необходимо создать или изменить.
- ☐ Определите все компоненты приложений, библиотек или оборудования, на которые повлияет изменение.
- ☐ Определите все стороннее ПО, которое потребуется приобрести или лицензировать.
- ☐ Определите влияние, которое предложенное изменение окажет на планы управления проектом ПО, проверки приемлемости качества, управления конфигурацией и другие планы.

Расчет затрат на изменение:

Часы	Задача
_____	Обновить спецификацию требований или базу данных требований.
_____	Разработать и оценить прототип.
_____	Создать новые компоненты дизайна.
_____	Изменить существующие компоненты дизайна.
_____	Разработать новые компоненты пользовательского интерфейса.
_____	Изменить существующие компоненты пользовательского интерфейса.
_____	Разработать новую пользовательскую документацию и экраны справки.
_____	Разработать новую пользовательскую документацию и формы справочной системы.
_____	Разработать новый исходный код.
_____	Изменить существующий исходный код.
_____	Приобрести и встроить стороннее ПО.
_____	Изменить файлы и процедуры сборки.
_____	Разработать новые модульные тесты и тесты интеграции.
_____	Изменить существующие модульные тесты и тесты интеграции.
_____	Выполнить модульные тесты и тесты интеграции после реализации.
_____	Написать новые системные и приемочные тесты.
_____	Изменить новые системные и приемочные тесты.
_____	Изменить автоматизированные средства тестирования.
_____	Выполнить регрессивное тестирование.
_____	Разработать новые отчеты.
_____	Изменить существующие отчеты.
_____	Разработать новые элементы базы данных.
_____	Изменить существующие элементы базы данных.
_____	Разработать новые файлы данных.
_____	Изменить существующие файлы данных.
_____	Изменить различные планы проекта.
_____	Обновить остальную документацию.
_____	Обновить матрицу связей требований.
_____	Просмотреть измененные продукты.
_____	Переработать продукт после рецензирования и тестирования.
_____	Другие задачи.
_____	Всего затрат

После чего можно сделать отчет об анализе влияния изменения:

Идентификатор запроса на изменение	_____
Название:	_____
Описание:	_____
Ответственный за оценку:	_____
Дата создания:	_____
Общая оценка затрат:	_____ рабочих часов
Оценка влияния на график:	_____ дней
Дополнительные затраты:	_____ долларов
Влияние на качество:	_____
Другие компоненты, затрагиваемые изменением:	_____
Другие задачи, затрагиваемые изменением:	_____
Проблемы стоимости жизненного цикла:	_____

ПРОБЛЕМЫ С ТРЕБОВАНИЯМИ

Проблемы также нужно фиксировать в отдельном документе, чтобы было сразу видно, где есть проблемы. Проблемные требования нужно разрешать своевременно.

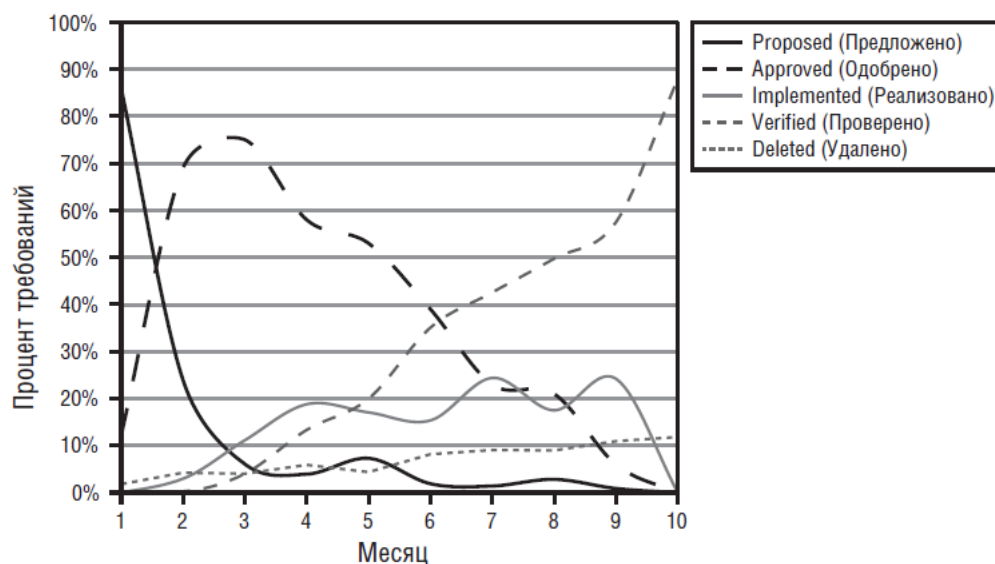
Тип проблемы	Описание
Вопрос по требованию	Что-то, что непонятно или не решено в отношении требования
Отсутствующее требование	В процессе реализации разработчики обнаружили отсутствующее требование
Неправильное требование	Требование неверно. Его нужно исправить или удалить
Вопрос по реализации	В процессе реализации у разработчиков возникают вопросы о том, как что-то должно работать или о вариантах дизайна
Дублирование требования	Обнаружены два или больше эквивалентных требований. Надо оставить одно, а остальные удалить
Ненужное требование	Требование просто больше не нужно

УПРАВЛЕНИЕ ТРЕБОВАНИЯМИ В AGILE

В Agile управление требований происходит в динамическом резерве пользовательских историй. Порядок внесения зависит от приоритета пользовательской истории. За содержание (т.е. то, что будет реализован в следующей итерации) отвечает владелец продукта или клиент.

Отслеживание состояний требований

Состояние требований нужно отслеживать, так как это дает наглядную картину процесса разработки:



Отслеживать затраченное время на работу с требованиями (подготовка, сбор, разработка, и т.д.). В дальнейшем это поможет планировать процесс формирования требований.

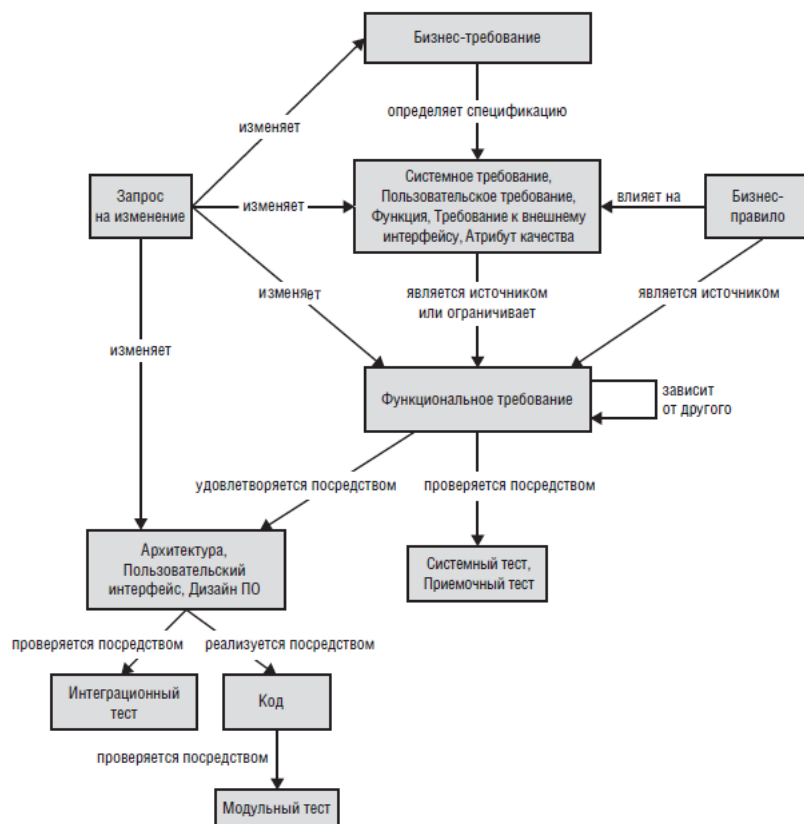
Возможные варианты состояний находятся в таблице ниже:

Состояние	Определение
Proposed (Предложено)	Требование запрошено уполномоченным лицом
In Progress (Разработка)	Бизнес-аналитик активно работает над требованием
Drafted (Подготовлено)	Написана начальная версия требования
Approved (Одобрено)	Требование проанализировано, его влияние на проект просчитано, и оно было размещено в базовой версии определенной версии. Ключевые заинтересованные в проекте лица согласились с этим требованием, а разработчики ПО обязались реализовать его
Implemented (Реализовано)	Код, реализующий требование, разработан, написан и протестирован. Определена связь требования с соответствующими элементами дизайна и кода. ПО, реализующее требование теперь готово для тестирования, рецензирования и другой проверки
Verified (Проверено)	Требование удовлетворяет критериями приемки, то есть подтверждено корректное функционирование реализованного требования. Определена связь требования с соответствующими вариантами тестирования. Теперь требование считается завершенным
Deferred (Отложено)	Одобренное требование теперь запланировано для реализации в более позднем выпуске
Deleted (Удалено)	Утвержденное требование удалено из базовой версии. Опишите причины удаления и назовите того, кто принял это решение
Rejected (Отклонено)	Требование предложено, но не запланировано для реализации ни в одном из будущих выпусков. Укажите причины отклонения требования и того, кто принял это решение

В Agile используется график сгорания задач: баллы истории против времени или итерации. У историй также отслеживается статус.

Отслеживание связей требований

ОБЩАЯ СХЕМА



ПЛЮСЫ ОТСЛЕЖИВАНИЯ

1. Обнаружение пропущенных требований (отсутствие связей между нужными требованиями).
2. Поиск ненужной функциональности (какое-либо требование не имеет связей).
3. Тестирование связанных блоков (проще искать ошибки).
4. Повторное использование связанных блоков (забирая какое-либо требование, проще определить, что нужно забрать вместе с ним).
5. Быстрое внесение изменений (лучше даже делать пометки в коде).
6. Можно удалить все требования, связанные с ненужным бизнес-требованием.

Не смотря на все плюсы, отслеживание требований – довольно затратное мероприятие. Не стоит вести отслеживание сразу через все уровни требований. Обычно достаточно отслеживать пары требований типа функциональные – пользовательские и т.д.

ИНСТРУМЕНТЫ ДЛЯ ОТСЛЕЖИВАНИЯ СВЯЗЕЙ ТРЕБОВАНИЙ

Матрица отслеживаемости требований. Показывает связь требований с кодом, функционалом, тестом и элементом дизайна.

Пользовательское требование	Функциональное требование	Элемент дизайна	Элемент кода	Тест
UC-28	catalog.query.sort	Каталог классов	CatalogSort()	search.7 search.8
UC-29	catalog.query.import	Каталог классов	CatalogImport() CatalogValidate()	search.12 search.13 search.14

Двусторонняя матрица отслеживаемости. Описывает связи одного типа требований с другими. Например, матрица UserCase – FuncReq:

Функциональное требование	Вариант использования			
	UC-1	UC-2	UC-3	UC-4
FR-1	←┘			
FR-2	←┘			
FR-3			←┘	
FR-4			←┘	
FR-5		←┘		←┘
FR-6			←┘	

Связи требований Источник-Цель отслеживаются и поддерживаются разными людьми (конечно, это все может делать и системный аналитик, но, все же, лучше, когда это делают специалисты). Аналитик должен координировать отслеживание:

Тип объекта источника ссылки	Тип объекта целевой ссылки	Источник информации
Системное требование	Функциональное требование	Системный инженер
Пользовательское требование	Функциональное требование	Бизнес-аналитик
Бизнес-требование	Пользовательское требование	Бизнес-аналитик
Функциональное требование	Функциональное требование	Бизнес-аналитик
Функциональное требование	Тест	Тестировщик
Функциональное требование	Элемент архитектуры	Архитектор или разработчик
Функциональное требование	Другие элементы дизайна	Дизайнер или разработчик
Элемент дизайна	Код	Разработчик
Бизнес-правило	Функциональное требование	Бизнес-аналитик

ОЦЕНКА РИСКОВ

Риски при разработке требований

- Риски нужно планировать заранее, так же, как и работу по их устранению, пока они не превратились в проблемы.
- Нужно периодически переоценивать риски.

Основные риски, связанные с требованием к ПО:

НА СТАДИИ ВЫЯВЛЕНИЯ ТРЕБОВАНИЙ

- Концепция и границы проекта (определить в начале проекта, чтобы потом опираться).
- Время, затраченное на разработку требований (все время отслеживать, сколько времени тратится на проект, чтобы правильно его оценивать).
- Вовлечение клиента (определить заинтересованные лица и классы пользователей, поддерживать контакт).
- Нефункциональные требования (не забывать про них).
- Единство мнения клиентов относительно требований к продукту.
- Несформулированные требования (которые клиенты считают очевидными – надо уметь извлекать и их тоже).
- Использование существующего продукта (но аккуратно, с учетом новизны и известных ошибок существующего продукта).
- Решения, предлагаемые под видом потребностей (аналитик должен найти причину, не всегда решение клиента – лучшее).

НА СТАДИИ АНАЛИЗА

- Определение приоритетов требований (важно, см. раздел выше).
- Технически сложные функции (оценка наиболее сложных для реализации функций, как вариант, создание прототипов для них,).
- Незнакомые языки, методы, среды разработки (обучение занимает довольно много времени).

НА СТАДИИ ДОКУМЕНТИРОВАНИЯ

- Понимание требований (тут поможет неформальное рецензирование коллег).
- Неоднозначная терминология (здесь можно создать словарь).
- Включение дизайна в требования (это дополнительные ограничения, которых, по-хорошему, не должны быть).

НА СТАДИИ УТВЕРЖДЕНИЯ

- Низкое качество проверки.
- Неутвержденные требования (это можно решить только проработкой и настойчивостью).

Риски при управлении требованиями

- Расползание границ (предотвратить расползание границ можно документов «О концепции и границах»; если базовая версия оказалось неполной, то использовать инкрементную методологию разработок).

- Процесс изменения требований (отсутствие анализа влияния изменения, информирование всех заинтересованных лиц).
- Нереализованные требования (важно отслеживать состояния требований).

ПРИЛОЖЕНИЕ А: МОДЕЛИРОВАНИЕ ТРЕБОВАНИЙ

Unified Modeling Language, UML

КЛАССИФИКАЦИЯ ДИАГРАММ

Диаграммы делятся на два больших типа:

1. Структурные
 - a. Диаграмма классов (Class Diagram) -
 - b. Диаграмма объектов (Object Diagram) – конкретные объекты на определенный момент времени.
 - c. Диаграмма пакетов (Package Diagram) – описывает группы классов и слои ПО (высокий уровень).
 - d. Диаграмма компонент (Component Diagram).
2. Поведенческие
 - a. Варианты использования (UseCase Diagram) – работа пользователей с системой.
 - b. Диаграмма активности (Activity Diagram) – как классические блок-схемы. Может быть уточнением каждого UseCase.
 - c. Диаграмма последовательности (Sequence Diagram) – объекты в действии. Как они взаимодействуют, в какой последовательности, время жизни.
 - d. Диаграмма состояний (State Machine Diagram).

STATE (MACHINE) DIAGRAM

Диаграмма (машина) состояний.

SEQUENCE DIAGRAM

Диаграмма последовательности - отображение жизненного цикла определенного объекта и взаимодействия с другими объектами в рамках определенного прецедента.

CLASS DIAGRAM

Используется в качестве ERD.

Используется при создании архитектуры ПО.

USECASE DIAGRAM

Основные элементы:

- Actor – пользователь, действующее лицо.
- UseCase – прецедент, цель, которую хочет достигнуть пользователь.
- Association – связь между элементами (актерами и целями).
- Include – включение, возможные реализации варианта.
- Extend – расширение, возможное дополнение варианта.

ПО ДЛЯ РАБОТЫ С UML

Icam DEFinition for Function Modeling, IDEF

СТАНДАРТЫ IDEF

IDEF0

Текст

IDEF1

IDEF1X

Business Process Modeling Notation, BPMN

Другие варианты

DATA FLOW DIAGRAM, DFD

Диаграмма потока данных – определяет преобразования системы, совокупность данных и материалов, которыми управляет система. Аналог - Activity Diagram UML, в котором можно еще включать логические операторы.

ENTITY-RELATIONSHIP DIAGRAM, ERD

Диаграмма «сущность-связь». Используется при проектировании баз данных. Для описания концептуальных схем. Нотация Chen (он первый и предложил ее). Может быть также в IDEF1X и Class Diagram UML (см. примеры выше).

Основные элементы диаграммы:

- Сущность (каждый блок описывается в единственном числе). У каждой сущности есть свои атрибуты. Обычно, каждая сущность потом становится таблицей в БД.
- Связи – логические соотношения между сущностями (в виде ромба в нотации Чена, в виде надписи и стрелки в UML).
- Мощность связи – количественное отношение. Например, М и 1 означает «много к одному».

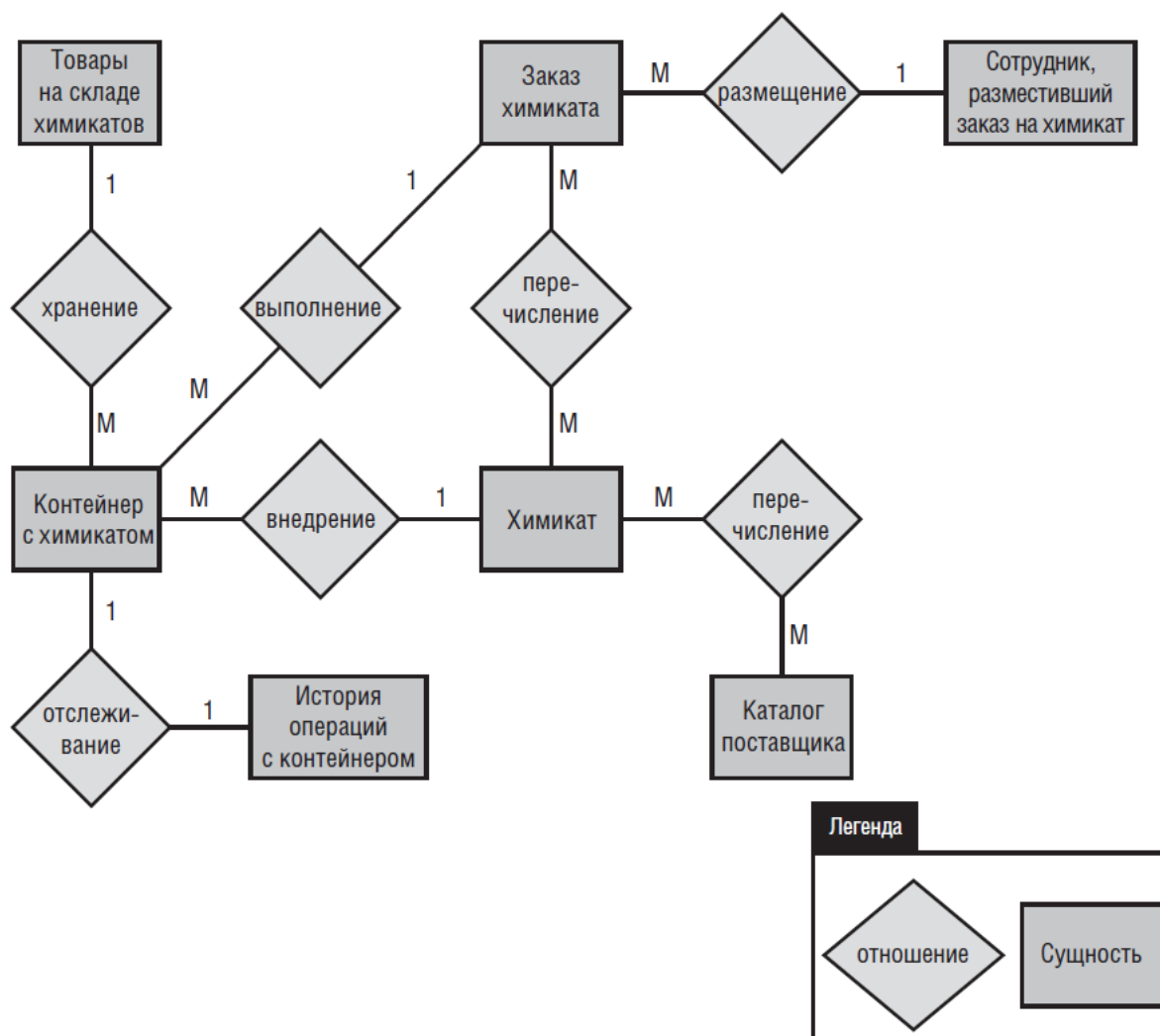


Диаграмма «сущность-связь» в нотации Чена.

STATE-TRANSITION DIAGRAM, STD

Диаграмма перехода состояний. Отображает возможные переходы между состояниями объекта или системы.

Альтернатива – таблица состояний.

Содержит три основных элемента:

- возможное состояние системы, в виде прямоугольников.
- разрешенное состояние (переходы), в виде стрелок.
- события или условия, вызывающий каждый переход, в виде текстовых пояснений.

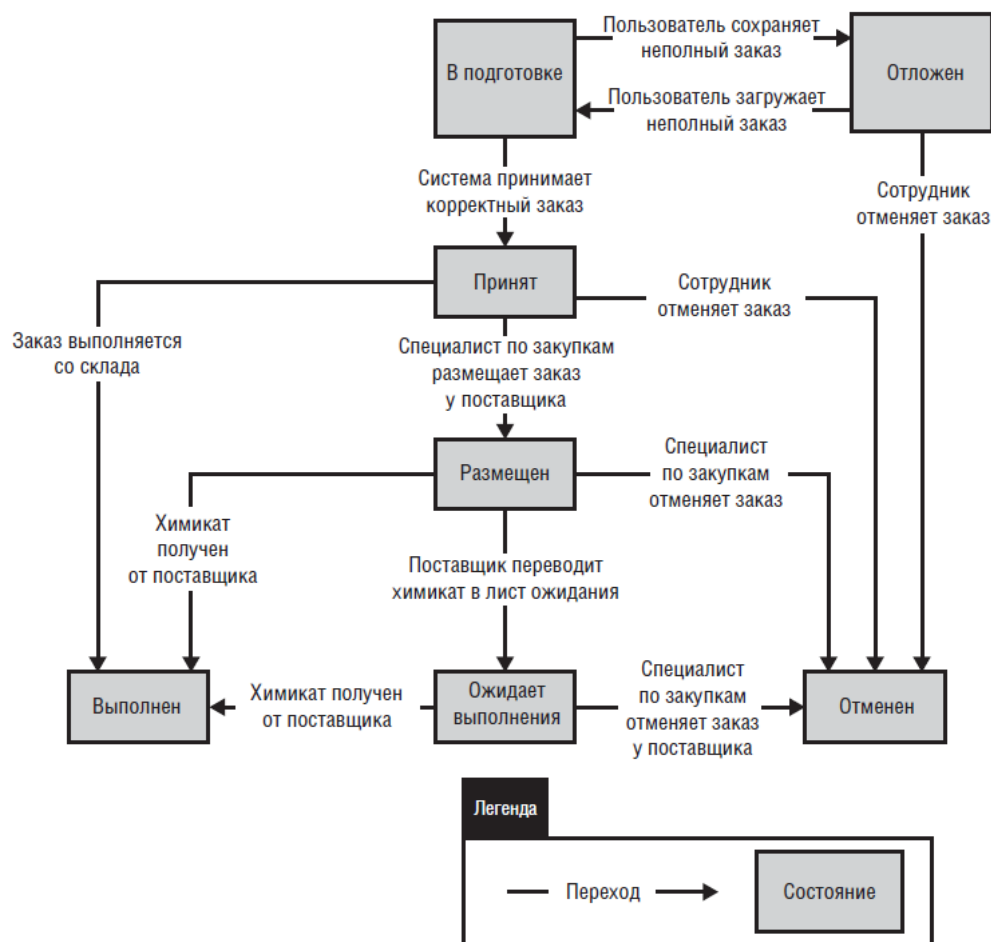


Диаграмма перехода состояний.

	В подготовке	Отложен	Принят	Размещен	Ожидает выполнения	Выполнен	Отменен
В подготовке	нет	Пользователь сохраняет неполный заказ	Система принимает корректный заказ	нет	нет	нет	нет
Отложен	Пользователь открывает неполный заказ	нет	нет	нет	нет	нет	нет
Принят	нет	нет	нет	Специалист по закупкам размещает заказ у поставщика	нет	Заказ выполняется со склада	Сотрудник отменяет заказ
Размещен	нет	нет	нет	нет	Поставщик переводит химикат в лист ожидания	chemical received from vendor	Специалист по закупкам отменяет заказ
Ожидает выполнения	нет	нет	нет	нет	нет	chemical received from vendor	Специалист по закупкам отменяет заказ
Выполнен	нет	нет	нет	нет	нет	нет	нет
Отменен	нет	нет	нет	нет	нет	нет	нет

Таблица перехода состояний.

SWIMLANE DIAGRAM

«Плавательная дорожка». Концепция. Описывает шаги, из которых состоит бизнес-процесс. Используется в BPMN и в Activity Diagram UML.

DESCISION TREE

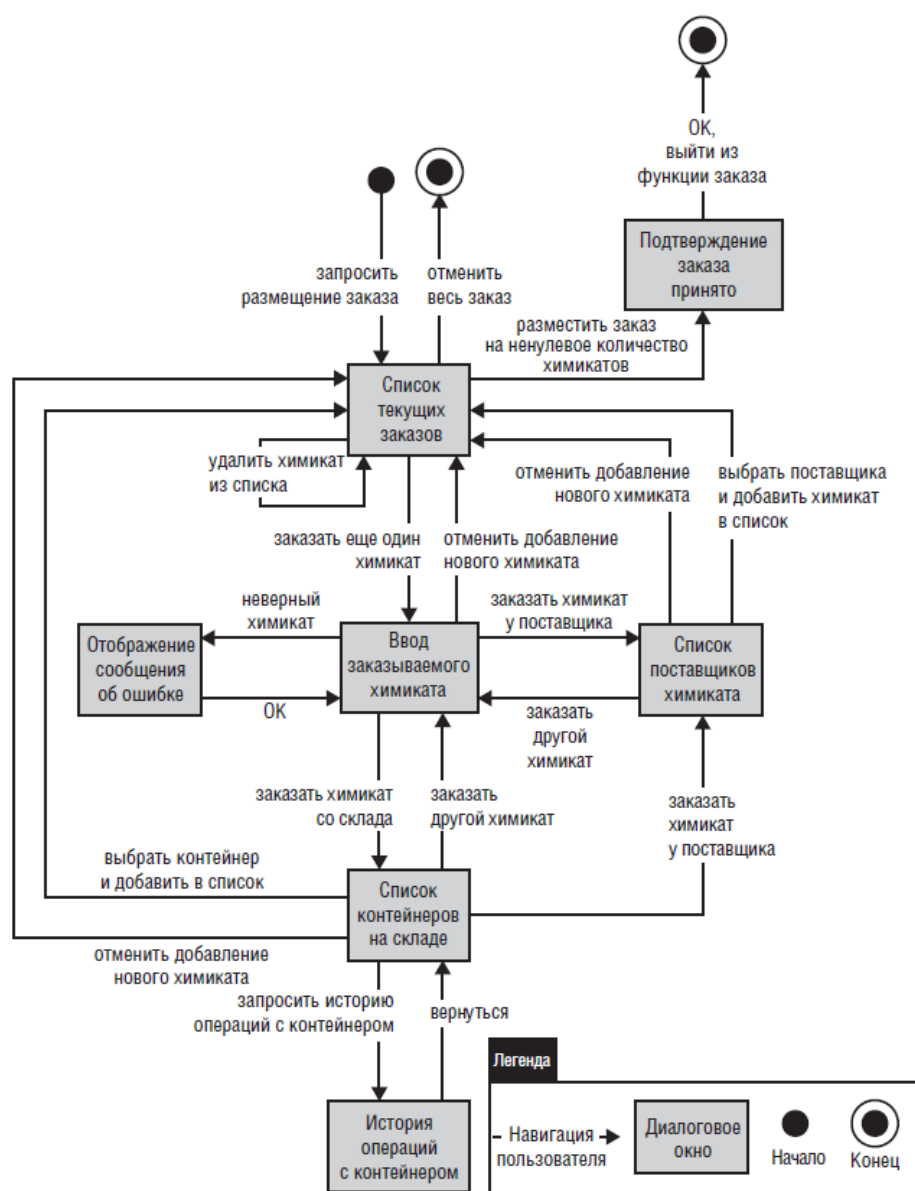
Древо решений. Концепция. Можно делать в Activity Diagram UML.

SYSTEM CONTEXT DIAGRAM

Контекстная диаграмма. Концепция. Описывает взаимодействие системы с внешней средой. Описывается IDEF0, можно использовать UseCase Diagram и Component Diagram UML.

КАРТЫ ДИАЛОГОВЫХ ОКОН

Вигерс. Представляет дизайн пользовательского интерфейса на высоком уровне. На ней отмечены элементы диалоговых окон в системе и ссылки навигации между ними, но не показан подробный дизайн экрана.



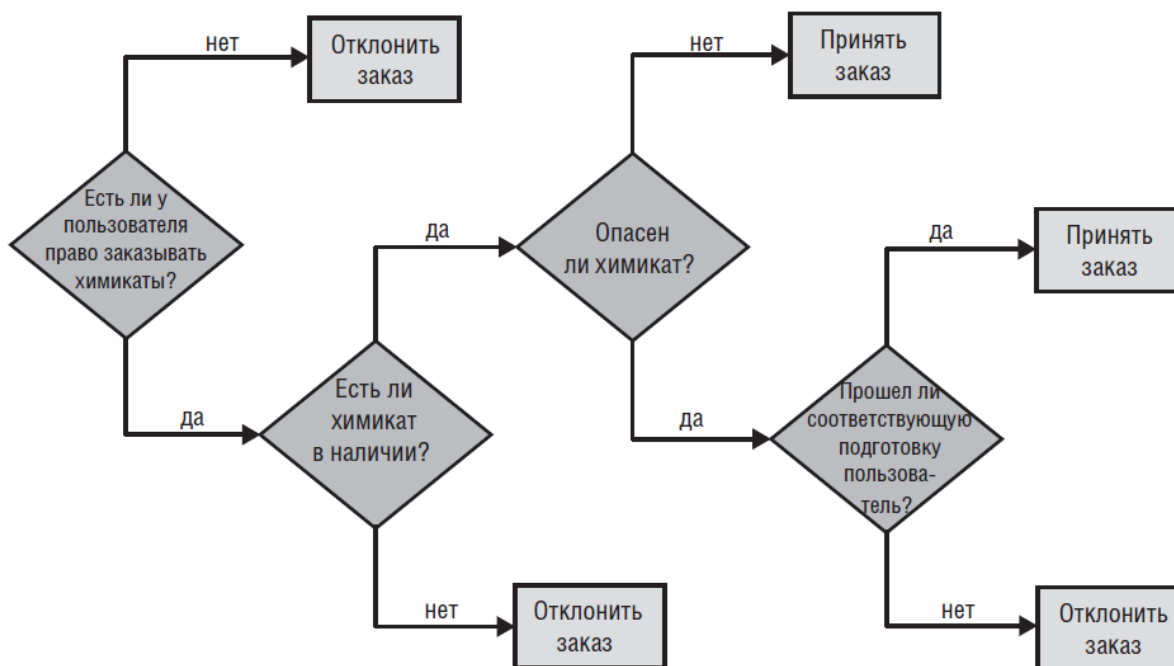
Карта диалоговых окон.

ТАБЛИЦА И ДРЕВО РЕШЕНИЙ

Удобны для сложных логических структур.

Номер требования					
Условие	1	2	3	4	5
Пользователь авторизован	false	true	true	true	true
Химикат есть в наличии	—	false	true	true	true
Химикат считается опасным	—	—	false	true	true
Сотрудник, разместивший заказ на химикат, прошел соответствующую подготовку	—	—	—	false	true
Действие					
Принять запрос			X		X
Отклонить запрос	X	X		X	

Таблица решений.



Древо решений.

ТАБЛИЦЫ СОБЫТИЙ И РЕАКЦИЙ

Удобны при создании систем реального времени и встроенного ПО. Аналог – Sequence Diagram

UML. Есть три типа событий:

- бизнес-событие – действие пользователя.
- сигнал – контрольный сигнал, полученный от внешнего устройства.
- временное событие – инициируется по времени.

